Journal of Software Engineering Research and Development
This work is licensed under a Creative Commons Attribution 4.0 International License.

HalluCodeDetection: Detecting Hallucinations in Code Generation

Rubens Nascimento de Lima

Abstract Small Language Models (SLMs) have gained increasing attention for code-generation tasks because their reduced computational requirements enable local execution, offering practical advantages in deployment cost, privacy, and data control. However, code generation remains susceptible to hallucinations, and this limitation may be particularly relevant for smaller models operating with more constrained capacity. More importantly, producing code and recognizing when generated code is incorrect are distinct capabilities that remain insufficiently understood for SLMs. In this work, we investigate whether SLMs can act not only as code generators but also as hallucination detectors, whether specialized training can improve this capability, and how their predictions compare with and benefit from deterministic static-analysis tools. We evaluate these questions on all 563 HumanEval+ and MBPP+ tasks from EvalPlus, using execution-based labels for Syntax, Runtime, Functional, and Correct outcomes. SLMs generate correct solutions for up to 80.8% of the tasks, reaching the same performance range as the substantially larger models evaluated in our comparison. Their predominant failures are not malformed syntax or execution crashes, but Functional hallucinations: programs that execute while deviating from the requested behavior. Error detection, however, follows a different pattern. No general-purpose model performs consistently well across all error categories, and Runtime hallucinations are particularly difficult, with the highest observed Runtime Recall reaching only 32.9%. Nevertheless, several SLMs detect more Runtime failures than Pyright, which identifies only 8.9% of the held-out Runtime cases under our rule mapping, showing that model-based reasoning can capture failures beyond those exposed by the selected static analysis. The two approaches are complementary: compilation provides deterministic Syntax detection, while incorporating tool feedback raises Syntax Recall to nearly 100% for most models. However, its effect on Runtime and Functional judgments remains model-dependent. Finally, specialized training substantially improves hallucination detection: an Accuracy-oriented detector gains 32.0 percentage points over its base model, while Macro-F1-oriented training produces a more balanced detector across error categories. Overall, our results show that code generation, hallucination detection, and correction are distinct capabilities, and that combining specialized SLMs with deterministic software-engineering tools provides complementary evidence for identifying failures in generated code.

Keywords: *Code Generation, Hallucination Detection, Specialized Fine-tuning, Small Language Models*

1 Introduction

Small Language Models (SLMs) have gained increasing attention for code-generation tasks because their lower computational requirements make local deployment more feasible, reducing infrastructure costs while providing greater control over data and privacy. These characteristics make SLMs attractive for programming assistants and other software-engineering applications in which source code may be sensitive or computational resources are limited. However, their more restricted model capacity does not eliminate the reliability problems associated with generative models. Generated code may appear plausible while still containing defects or deviating from the requested behavior, a phenomenon commonly discussed as *code hallucination* (Pavel et al., 2025; Lee et al., 2025). As SLMs become increasingly capable code generators, understanding whether they can also recognize problems in generated code becomes important for assessing their reliability beyond generation performance alone.

Detecting these problems is difficult because a hallucination and its observable consequence are not necessarily the same concept. In this work, we use *hallucination* to describe the generative phenomenon that leads a model to produce plausible but incorrect code, while the resulting failures are operationalized as observable error outcomes. Following prior work on code-generation failures (Lee et al., 2025; Tian et al., 2025), we distinguish *Syntax Error*, *Runtime Error*, and *Functional Error*, together with *Correct* solutions. Syntax Errors can often be identified mechanically, whereas Runtime Errors may emerge only under particular execution conditions, and Functional Errors may remain hidden because the program executes normally while producing behavior inconsistent with the specification. Test execution can provide objective evidence when sufficiently comprehensive tests are available, but such information is not necessarily accessible to a model during code assessment. Moreover, generating correct code and recognizing errors are different tasks, so strong generation performance does not necessarily imply strong evaluation capability.

Our central idea is therefore to study SLMs beyond their role as code generators and evaluate whether they can also serve as code-error detectors. We investigate whether general-purpose SLMs can recognize different categories of errors, whether they can identify errors in code they generated themselves, and whether task specialization can improve this capability without increasing model scale. We further compare model-based detection with deterministic static-analysis techniques to understand which types of errors each approach can identify and whether their signals are complementary. Finally, we examine whether external static-analysis evidence can help SLMs both classify gener-

ated code and revise their own solutions, and we compare these behaviors with those observed in substantially larger language models.

To operationalize this investigation, we use all 563 programming tasks from HumanEval+ and MBPP+ provided by EvalPlus (Liu et al., 2023). Multiple SLMs first generate solutions, which are compiled and executed against the EvalPlus test suites to derive an execution-based reference indicating *Syntax Error*, *Runtime Error*, *Functional Error*, or *Correct*. Runtime and Functional Errors may co-occur when different test cases expose different behaviors. The generated programs are then presented to language models without test results or reference labels, turning error detection into an independent multilabel classification task. The study is organized around seven questions: how well SLMs generate correct code; how well they detect errors and evaluate their own outputs; whether specialized SLMs improve error detection; how deterministic static-analysis techniques compare with SLM-based detection; how static-analysis feedback affects error detection; how an iterative static-analysis feedback cycle affects code generation; and how these behaviors compare with substantially larger language models. For specialization, we construct supervised training data from execution-derived labels and teacher-generated explanations and fine-tune Gemma-based models. Static-analysis signals are evaluated independently, as supporting evidence during error detection, and as feedback during iterative code generation.

The results reveal a clear separation between generation and error detection. The evaluated SLMs generate correct solutions for up to 80.8% of the EvalPlus tasks, with several models reaching the same observed performance range as the two substantially larger models used for comparison. Their predominant failures are Functional Errors rather than Syntax Errors, showing that the main difficulty is often not producing executable Python code, but producing code that fully satisfies the requested behavior. Error detection is substantially more heterogeneous. No general-purpose model performs consistently well across all categories, and Runtime Errors are particularly difficult, with the highest observed Runtime Recall reaching only 32.9%. Even under this limitation, some SLMs identify more Runtime Errors than the static-analysis technique evaluated for this category, which detects only 8.9% of the held-out Runtime cases. Static analysis is considerably stronger for syntactic failures, and providing this evidence to the models raises Syntax Error detection to nearly perfect levels for most evaluated models, although its effects on Runtime and Functional decisions remain model-dependent. Specialization further improves detection: the Accuracy-oriented detector gains 32.0 percentage points over its base model, while Macro-F1-oriented training produces a more balanced profile across error categories. Together, these results indicate that generation, error recognition, interpretation of external evidence, and code revision represent related but distinct capabilities.

The main contributions of this work are:

- An empirical evaluation of SLMs for both code generation and error detection under a common execution-based reference, distinguishing Syntax, Runtime, and Functional Errors through multilabel classification.
- An analysis of *self-error detection*, investigating whether models can recognize errors in code they generated themselves and examining the relationship between generation and evaluation capabilities.
- A specialized SLM-based detector trained with execution-derived labels and teacher-generated explanations, together with an analysis of how different optimization objectives affect the detector's behavior across error categories.
- An investigation of the relationship between model-based reasoning and deterministic static analysis, evaluating static techniques independently, as supporting evidence for error detection, and as feedback during iterative code generation.
- A comparison between SLMs and substantially larger language models under the same generation, detection, and tool-assisted protocols, showing that selected SLMs can occupy the same observed performance range while requiring substantially fewer parameters.

The remainder of this paper is organized as follows. Section 2 reviews previous work on code hallucinations, error characterization, automatic evaluation, specialized detectors, and feedback-guided code generation. Section 3 describes the experimental design, evaluated models, execution-based reference construction, specialized training, and integration of static-analysis techniques. The results section addresses the seven research questions concerning code generation, error detection, specialization, static-analysis baselines, static-analysis-assisted detection, iterative tool-assisted generation, and comparison with substantially larger models. The paper then discusses threats to validity and concludes by summarizing the main findings and outlining directions for future work.

2 Related Work

2.1 Hallucinations and error taxonomies in code generation

The term *hallucination* has no single, universally accepted operational definition in code generation. Pavel et al. (2025) characterize hallucinations as outputs that appear plausible but are factually incorrect, unverifiable, or nonsensical, and report manifestations such as syntax violations, invalid references, and conflicts with API knowledge. This perspective is particularly relevant to code generation because syntactically plausible source code may still fail during execution or behave differently from the requested specification.

Different studies have proposed taxonomies to characterize these failures. Lee et al. (2025) organize code-generation hallucinations into syntactic, runtime-execution, functional-correctness, and code-quality categories. Dou et al. (2025) provide another important perspective by organizing failures in generated code into three broad bug categories and ten subcategories. Their taxonomy was an important motivation for the classification adopted in this work because it reinforces that generated-code failures should not be treated as a single undifferentiated notion of incorrectness. CodeHalu pro-

vides a different decomposition, grouping hallucinations into mapping, naming, resource, and logical categories through execution-based verification (Tian et al., 2025).

These taxonomies characterize generated-code failures from different perspectives, including their source, manifestation, and observable effect. In this work, we distinguish the hallucination phenomenon from the error produced as its observable consequence. We do not attempt to infer the underlying cause of a hallucination from the generated program. Instead, we operationalize its effects as *Syntax Error*, *Runtime Error*, and *Functional Error*, together with *Correct* for solutions that satisfy the available evaluation suite. This outcome-oriented abstraction is intentionally coarser than some previous taxonomies, but it enables each category to be grounded consistently in compilation and execution behavior. Code-quality issues are outside the scope of this study because our focus is correctness and executability rather than style, maintainability, or efficiency.

2.2 Empirical evidence on failures in generated code

Empirical studies show that failures in generated code extend well beyond compilation problems. Dou et al. (2025) evaluate closed- and open-source models across multiple benchmarks and show that error distributions depend on both the evaluated model and the characteristics of the programming tasks. Their results also indicate that real-world tasks can exhibit failure distributions different from those observed in conventional benchmarks. Similarly, Pavel et al. (2025) show that generated-code failures can depend on domain knowledge and API constraints, even when additional contextual information is provided to the model.

These findings motivate analyzing error categories explicitly rather than relying only on an aggregate correctness measure. However, characterizing the errors produced by a generator does not determine whether a model can recognize those same errors when examining generated code. This distinction is central to our study. We evaluate generation and error detection under the same execution-based reference to investigate whether models that generate code successfully are also capable of distinguishing Correct solutions from Syntax, Runtime, and Functional Errors. We further examine this relationship for self-evaluation, where a model is asked to identify errors in code that it generated itself.

2.3 Automatic evaluation and error detection

Execution against tests provides a direct mechanism for identifying many observable failures in generated programs. EvalPlus strengthens HumanEval and MBPP with additional automatically generated tests and demonstrates that the original test suites can accept incorrect solutions and affect the relative assessment of code-generation models (Liu et al., 2023). This motivates the use of strengthened execution-based evaluation when constructing reference labels. In our study, compilation and EvalPlus execution determine the reference error outcomes, while the language models evaluated as detectors do not receive these execution results.

CodeHalu follows a related execution-based direction and introduces CodeHaluEval, containing 8,883 generated programs from 699 tasks (Tian et al., 2025). Its objective, however, differs from ours in both taxonomy and detection procedure. CodeHalu characterizes hallucinations using its own error taxonomy and an execution-based heuristic pipeline, whereas HalluCodeDetection formulates error detection as a multilabel classification problem over observable Syntax, Runtime, Functional, and Correct outcomes.

The detector-oriented work most closely related to ours is Andriushchenko et al. (2026), who construct line-level hallucination annotations and train a lightweight Transformer detector using internal representations from language models. Their results demonstrate that specialized detectors can identify hallucinated regions and outperform generic uncertainty-based approaches. Our work investigates a different granularity and setting. Rather than locating individual hallucinated lines, we ask whether general-purpose and specialized SLMs can classify the observable failure mode of a complete generated solution. We additionally study self-error detection, class-specific behavior, and the interaction between model-based detection and external software-analysis evidence.

2.4 Specialized evaluators and rationale-based supervision

Learned evaluators have previously been incorporated into code-generation systems. CodeRL trains a critic to estimate functional correctness and uses critic predictions together with unit-test feedback to improve a code generator through reinforcement learning (Le et al., 2022). Its outcome space includes compilation errors, runtime errors, failed tests, and passed tests, making it an important precedent for distinguishing different observable execution outcomes. However, the critic in CodeRL is primarily a component used to optimize generation rather than a stand-alone multilabel detector evaluated across different error categories.

Our specialized detector instead treats error recognition as the primary task. We fine-tune SLMs to predict the reference error categories directly and investigate how optimization objectives affect the resulting decision behavior. The detector is trained not only from execution-derived labels but also from natural-language explanations produced by teacher models. This design follows previous work showing that rationale-based supervision can provide richer training signals than labels alone. Hsieh et al. (2023) use generated rationales as additional supervision when distilling knowledge into smaller models, while Ludan et al. (2023) study explanation-based fine-tuning as a mechanism for reducing reliance on spurious cues. In HalluCodeDetection, the teacher-generated explanations serve only as auxiliary supervision: the reference labels remain determined by execution, and detector performance is evaluated exclusively from the predicted error classes.

The distinction from previous critic-based systems is therefore not the general idea of using a model to assess generated code. Rather, we investigate whether an SLM can be specialized as an independent multilabel error detector and whether specialization changes its ability to distinguish individual error categories, particularly under a strongly imbalanced class distribution.

2.5 Self-evaluation, self-correction, and tool-assisted generation

Several approaches improve model outputs through iterative feedback. Self-Refine alternates between generation, self-feedback, and refinement without additional model training (Madaan et al., 2023), while Reflexion uses verbal feedback from previous attempts to guide subsequent decisions and includes programming tasks among its applications (Shinn et al., 2023). These approaches demonstrate that models can sometimes use their own assessments to revise an output, but successful revision does not necessarily imply that the underlying error was identified correctly.

For code generation, Self-Debugging asks a model to inspect execution information and reason about its program before producing a revised solution (Chen et al., 2024). Dou et al. (2025) similarly investigate a training-free correction process based on self-critique, bug information, and compiler feedback. These studies establish that external evidence and iterative revision can improve code-generation workflows. Our study separates three related capabilities that are often considered together: recognizing an error, interpreting external evidence about that error, and successfully revising the program.

This distinction also motivates our comparison with deterministic software-analysis techniques. Compilers and static analyzers can provide direct evidence for some classes of errors, while language models may reason about behaviors that are not captured by a particular static analysis. We therefore evaluate static techniques independently, use their outputs as supporting evidence for model-based classification, and provide the same evidence during iterative code generation. This allows us to examine not only whether an error signal exists, but also whether an SLM can interpret and act on that signal appropriately.

Taken together, prior work has established taxonomies for code hallucinations, execution-based evaluation, specialized critics, self-correction, and tool-assisted generation. HalluCodeDetection builds on these directions but studies a combination that remains less explored for SLMs: generation and error detection under the same execution-based reference, self-error detection, specialization as a stand-alone multilabel detector, and the complementarity between learned detection and deterministic software-analysis evidence. This design allows us to investigate whether generation, error recognition, evidence interpretation, and correction behave as the same capability or as distinct aspects of reliable code generation.

3 Methodology

3.1 Study Design and Research Questions

This study investigates the ability of Small Language Models (SLMs) to both generate correct code and identify errors in generated code. The study combines code generation, execution-based classification, error detection, model specialization, and static-analysis assistance to examine these capabilities under a common evaluation framework.

Several language models generate solutions for standardized programming tasks. Each generated solution is compiled and executed against a test suite to determine whether it is correct or exhibits observable errors. These execution-derived outcomes serve two purposes: they provide the reference for evaluating code-generation capability and constitute the ground truth for the subsequent error-detection experiments.

Error detection is evaluated separately from code generation. In this setting, the model receives the programming task description and the generated code, but does not receive reference labels or test execution results. The detection task is formulated as multilabel classification because a generated solution may exhibit more than one observable error type across different test cases. We additionally investigate whether models can identify errors in code that they generated themselves and whether specialization improves error-detection capability.

The study is organized around seven research questions:

- **RQ1:** To what extent can different language models generate correct code?
- **RQ2:** To what extent can SLMs correctly identify errors in generated code?
- **RQ3:** To what extent can specialized SLMs improve the detection of errors in generated code?
- **RQ4:** How do static-analysis techniques compare with SLMs in detecting errors in generated code?
- **RQ5:** How does static-analysis feedback affect the ability of language models to detect errors in generated code?
- **RQ6:** How does an iterative static-analysis feedback cycle affect the ability of language models to generate correct code?
- **RQ7:** How do SLMs compare with substantially larger language models across code generation, error detection, and tool-assisted settings?

The main experimental factors are the evaluated model, the specialization strategy, and the availability of static-analysis feedback. Direct comparisons keep the programming tasks, error taxonomy, prompt structure, output schema, evaluation metrics, and evaluation harness fixed whenever applicable. Model family and pretraining data necessarily vary across models, while backend, numerical precision, quantization, and hosted-model revision are not fully controlled. Cross-model findings are therefore interpreted as benchmark-conditioned observations rather than as causal effects of parameter count.

3.2 Benchmark and Unit of Analysis

The experiments use the EvalPlus benchmark (Liu et al., 2023), which extends HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021) with additional test cases intended to strengthen correctness assessment. EvalPlus provides 164 HumanEval+ tasks and 399 MBPP+ tasks, totaling 563 programming tasks.

We use all 563 tasks available in these two EvalPlus portions rather than selecting a subset. This avoids sampling

tasks after observing model behavior, but does not make the benchmark representative of all programming domains. HumanEval+ and MBPP+ are suitable for this study because they provide executable Python function-level tasks and strengthened test suites under a common evaluation harness. At the same time, the conclusions remain conditioned on the benchmark, programming language, task complexity, model families, prompting strategy, and evaluated tools.

Each task contains a natural-language specification describing the required functionality. The tasks include common algorithmic problems involving strings, numerical operations, collections, and data structures. The programming task is the independent unit of analysis. Multiple generated solutions, predictions, or rationale-expanded records derived from the same task share the same specification and test suite and are therefore treated as dependent observations rather than as independent samples.

3.3 Ground Truth and Error Classification

In this work, a *hallucination* refers to the generative phenomenon in which a model produces plausible but incorrect code. The evaluation does not attempt to infer the underlying cause of a hallucination. Instead, it operationalizes its observable consequences as *Syntax Error*, *Runtime Error*, and *Functional Error*. *Correct* represents the non-error outcome when no observable failure is detected by the available evaluation suite.

The ground truth is determined by compiling and executing each generated solution against a custom harness built on the EvalPlus tests. Labels are accumulated across test cases, which allows Runtime and Functional Errors to co-occur when different tests expose different behaviors. Syntax Error and Correct are exclusive outcomes.

The procedure begins with compilation of the candidate source produced after response normalization. If the source cannot be compiled, the solution receives the Syntax Error label and evaluation stops. This category therefore includes genuine Python syntax errors as well as cases in which the generated response or extraction process produces source code that cannot be compiled, such as incomplete structures or incompatible output formatting.

If the source compiles successfully, it is executed in an isolated subprocess against the available test cases. For each test case, an exception or timeout produces a Runtime Error. If execution completes but the returned value differs from the expected output, a Functional Error is recorded. Runtime and Functional Errors may therefore co-occur in the same solution when different test cases expose different failures.

The Correct label is assigned only when the candidate compiles successfully, no test case produces an exception or timeout, and all observed outputs match the expected results. A Correct solution cannot receive any other label. Algorithm 1 summarizes this procedure.

Algorithm 1 Execution-based ground-truth labeling procedure

Require: Generated candidate source and EvalPlus tests
Ensure: Set of outcome labels L

$L \leftarrow \emptyset$
if candidate does not compile **then**
  $L \leftarrow L \cup \{\text{Syntax Error}\}$
  return L
end if
for each test case **do**
  execute candidate
  if execution raises an exception or exceeds the timeout **then**
    $L \leftarrow L \cup \{\text{Runtime Error}\}$
  else if observed output $\neq$ expected output **then**
    $L \leftarrow L \cup \{\text{Functional Error}\}$
  end if
end for
if $L = \emptyset$ **then**
  $L \leftarrow \{\text{Correct}\}$
end if
return L

Table 1 summarizes the operational definitions used throughout the study.

Table 1. Operational outcome categories used in the study

Outcome	Operational definition
Syntax Error	The candidate source cannot be successfully compiled after response normalization.
Runtime Error	The candidate compiles, but execution raises an exception, exceeds the timeout, or otherwise fails during a test case.
Functional Error	The candidate executes without a Runtime Error for a test case but produces an output different from the expected result.
Correct	The candidate compiles, produces no Runtime Error, and returns the expected result for every available test case.

These categories characterize observable outcomes rather than the root cause of a generated-code hallucination. In particular, passing all EvalPlus tests is interpreted as correctness with respect to the available evaluation suite and not as a formal proof of correctness for all possible inputs.

3.4 Evaluation Metrics

The study uses metrics according to the capability evaluated by each research question.

For code generation, correctness is measured as the proportion of the 563 tasks for which the generated solution receives the Correct label according to the execution-based reference.

For error detection, the task is evaluated as multilabel classification over Syntax Error, Runtime Error, Functional Error, and Correct. For each outcome independently, true positives (TP), false positives (FP), and false negatives (FN) are

computed.

Per-class recall is defined as

$$\text{Recall} = \frac{TP}{TP + FN},$$

and measures the proportion of reference cases of a given outcome that are correctly identified. Per-class precision is defined as

$$\text{Precision} = \frac{TP}{TP + FP},$$

and measures the proportion of predictions of a given outcome that correspond to the reference label.

The F1 score for each class is calculated as

$$F1 = \frac{2TP}{2TP + FP + FN}.$$

Macro-F1 is the unweighted mean of the F1 scores across the four outcome categories and therefore gives equal contribution to frequent and rare classes.

Exact-set Accuracy measures the proportion of samples for which the complete set of predicted labels exactly matches the execution-based reference set. Partially correct multilabel predictions are therefore not counted as exact matches.

Self-error detection is evaluated only on erroneous solutions generated by the same model being assessed. For each model, Self-Detection is the proportion of its erroneous generated solutions for which the predicted set of error labels exactly matches the execution-based reference. Per-class self-detection rates are additionally reported for the individual error categories when the model produced at least one reference case of that category.

3.5 Statistical Analysis

The statistical analysis is defined according to the objective of each research question. We use inferential procedures only when they directly support a comparison required by the corresponding RQ. The programming task is treated as the independent unit of analysis, and paired analyses compare results obtained for the same tasks.

RQ1 and RQ2 are analyzed descriptively. RQ1 characterizes code-generation capability using the proportion of solutions classified as Correct and the distribution of Syntax, Runtime, and Functional Errors for each model. RQ2 characterizes error-detection capability using Exact-set Accuracy, Macro-F1, per-class Recall, and Self-Detection. These research questions are intended to describe the observed capabilities and error profiles of the evaluated models rather than test whether every pair of models differs statistically.

For RQ3, the main comparison concerns whether specialization improves error detection relative to the corresponding base model. We therefore report the paired difference in Accuracy and Macro-F1 between each selected specialized detector and its base condition. A paired task-level bootstrap is used to estimate an interval for these differences while preserving the pairing between predictions obtained for the same programming tasks. This analysis quantifies both the magnitude of the observed improvement and its variability across the evaluated tasks.

RQ4 is descriptive. The standalone performance of Python compilation and Pyright is reported using the number and proportion of reference errors detected for the outcome categories to which each technique is mapped. These values are compared with the corresponding per-class recall observed for the language-model detectors. Because the tools and models use different detection mechanisms and the static techniques do not cover all outcome categories, this analysis is interpreted as a comparison of observable detection coverage rather than as an omnibus ranking.

RQ5 is also descriptive. The effect of providing static-analysis feedback to the detectors is examined through changes in Exact-set Accuracy, Macro-F1, and per-class Recall relative to the no-feedback condition of RQ2. The analysis focuses on whether the additional evidence changes the detection of *Syntax Error* and *Runtime Error*, which are directly related to the tools, and whether it also changes predictions of *Functional Error* and *Correct*.

RQ6 evaluates the complete iterative generation protocol relative to the original generation condition of RQ1. Changes in the proportion of *Correct*, *Functional Error*, *Runtime Error*, and *Syntax Error* outcomes are reported descriptively for all models. Where paired per-task observations from both protocols are available, an exact two-sided McNemar test is used to assess whether the proportion of *Correct* outcomes changes systematically. The original and iterative protocols differ not only by the availability of static-analysis feedback but also by decoding temperature, self-review, and repeated revision; the comparison is therefore interpreted as a comparison between complete protocols rather than as an isolated causal effect of static analysis.

RQ7 is descriptive. The results obtained for SLMs are compared with those of the substantially larger models across generation, error detection, and tool-assisted settings. No additional significance test is applied to model scale because parameter count varies together with model family, architecture, training, numerical precision, and backend. The comparison is therefore interpreted only for the evaluated models and experimental conditions.

4 Experimental Setup

4.1 Models and Inference Configuration

The study evaluates language models with parameter counts ranging from 4B to 552B, as shown in Table 2. The table separates the models treated as SLMs in this study from two substantially larger hosted models used as comparison baselines.

The local SLMs used in the initial code-generation experiment were executed with Q4_K_M quantization, which allowed larger local models to be evaluated within the available computational budget. The Gemma models were also evaluated without precision reduction in the detection experiments. Fine-tuned models were executed without inference quantization, while their adaptation used QLoRA with 4-bit

Table 2. Models evaluated in the study.

Model	Parameters
Small language models	
Devstral Small 2	24B (Rastogi et al., 2025) (Mistral AI, 2025)
GLM 4.7 Flash	30B (GLM-4.5 Team et al., 2025) (Z.ai, 2025)
Gemma 3 4B	4B (Gemma Team et al., 2025)
Gemma 4	{5.1, 8, 31}B (Gemma Team et al., 2026)
GPT-OSS 20B	20B (OpenAI et al., 2025)
Muse Glimmer 30B	30B (Meta Superintelligence Lab, 2026)
Ornith 1.5 35B	35B (Ornith Team, 2026)
Qwen 2.5 Coder	{7, 32}B (Hui et al., 2024)
Qwen 3 Coder 30B	30B (Yang et al., 2025) (Qwen Team, 2025)
Qwen 3.8 27B	27B (Qwen Team, 2026)
Large models used for comparison	
DeepSeek V4.1 Flash	552B (DeepSeek-AI, 2026)
MiMo V2.5	310B (Xiaomi MiMo Team, 2026)

NF4 quantization (Dettmers et al., 2023). This training quantization is distinct from the Q4_K_M quantization used for local inference. The two substantially larger comparison models were accessed through hosted APIs, and no quantization level is assumed for them.

Three models were used as judges during knowledge distillation: Muse Glimmer 30B, Qwen 3.8 27B, and Ornith 1.5 35B. Gemma 4 E2B, Gemma 4 E4B, and Gemma 3 4B were used as base models for specialized detector training.

The initial code-generation experiment uses a temperature of 1.0. Knowledge distillation, error detection, tool-assisted detection, and the iterative generation protocol use a temperature of 0.0 to minimize sampling variability. Across the evaluated conditions, models operate in direct-response mode, with thinking or reasoning controls disabled whenever the corresponding backend exposes such options. No separate reasoning trace is retained or supplied as input to subsequent stages.

4.2 Code Generation Experiment

The code-generation experiment evaluates each model on the 563 programming tasks described in Section 3.2. A system prompt instructs the model to act as a Python programming assistant and to return a single function implementation without tests, assertions, or example usage. For HumanEval+, the model receives the function signature and is asked to complete the function body. For MBPP+, the model is asked to generate the complete function implementation.

Each model produces one solution per programming task using a temperature of 1.0. The top-p and top-k parameters remain at the defaults provided by the corresponding model or inference backend, and no fixed generation seed is imposed.

Before execution, the raw model response is normalized into candidate source code. Markdown code fences are removed when they wrap the complete response, and the extraction procedure determines whether the model returned a complete entry-point function or only a function body. For HumanEval+ tasks, the benchmark prompt already contains the function header. When only a body is returned, the normalized body is appended to this header; when a complete function definition is returned, that definition is used directly. For MBPP+, a complete returned function is used when available; otherwise the required signature is added and the returned body is indented accordingly.

The normalization procedure does not remove arbitrary natural-language content that appears before or after the generated code. Consequently, responses that violate the requested output format may remain uncompilable and receive an *Syntax Error* under the execution-based reference defined in Section 3.3. Similarly, extraction failures that produce incomplete candidate source are preserved rather than silently corrected.

4.3 Error Detection and Self-Evaluation Experiment

The error-detection experiment evaluates whether language models can identify observable errors in generated code without access to reference labels or test execution results. For each sample, the detector receives only the programming-task description and the generated code.

The system prompt defines the classification task using the outcome categories introduced in Section 3.3: *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*. The model is instructed to return a structured object containing the detected outcomes together with a natural-language explanation for each prediction. The explanations must remain general and must not refer to hidden test cases or specific test inputs.

All models evaluated as detectors follow the same protocol. They receive the same task description, generated code, class definitions, prompt structure, output schema, and temperature. No test results or execution-derived labels are provided during inference.

The detection experiments reported in RQ2 are evaluated exclusively on the test partition defined for specialized detector training. This partition contains 113 of the 563 programming tasks, corresponding to approximately 20% of the benchmark. No task assigned to this test partition is used for specialized-model training or validation.

If a detector response cannot be parsed into the expected structured output, its prediction is represented as an empty set of labels. The sample is not discarded and therefore remains part of the evaluation. This prevents malformed detector responses from being excluded from the reported metrics.

Self-evaluation uses the same test partition and does not require an additional inference procedure. For each generator model, the analysis selects the test samples whose code was generated by that same model and compares its detection predictions with the execution-derived reference. Self-Detection therefore measures whether a model can recognize

the observable errors present in its own generated code under the same detection protocol used for the other samples.

4.4 Specialized Detector Training

4.4.1 Knowledge Distillation

The specialized detector is trained using execution-derived labels together with natural-language explanations generated through knowledge distillation. For each generated solution, a judge receives the programming-task description, the generated code, the identity of the generator model, and the reference error labels previously determined through execution.

The judge does not determine or modify the reference label. Instead, it produces an explanation describing why the code may exhibit each supplied error outcome without referring to specific hidden test cases or input values. The execution-derived label therefore remains authoritative, while the judge-generated explanation acts only as auxiliary supervision.

Each generated solution is processed independently by Muse Glimmer 30B, Qwen 3.8 27B, and Ornith 1.5 35B. Each judge produces its own rationale for the same code and reference label. No voting or judge-selection procedure is performed. Consequently, a generated solution contributes one training record per judge.

The resulting rationale-expanded dataset contains 18,579 records derived from 6,193 generated solutions. If a judge produces an explanation that cannot be parsed into the expected structure, the execution-derived label is preserved and the raw returned text is retained as the rationale. If the judge call itself fails and produces no usable response, the sample is retried in a later execution.

The rationale is included to provide supervision beyond the class name alone. Label-only supervision indicates which prediction is expected but provides limited explicit information about the relation between the task specification, the generated implementation, and the corresponding error outcome. Rationale-based supervision instead associates the target prediction with an explanation grounded in the generated code. This design follows prior work on rationale-based distillation (Hsieh et al., 2023) and explanation-based fine-tuning (Ludan et al., 2023). The explanations are treated only as auxiliary training targets. Quantitative evaluation considers the predicted classes and does not score the textual quality of the explanations.

4.4.2 Dataset Partition and Class Distribution

The rationale-expanded dataset is partitioned at the programming-task level into training, validation, and test sets. All generated solutions associated with the same programming task, including solutions produced by different models and all rationale-expanded records produced by the judges, are assigned to the same partition.

The partition contains 366 training tasks, 84 validation tasks, and 113 test tasks, corresponding approximately to 65%, 15%, and 20% of the 563 programming tasks. Table 3 summarizes the resulting partitions.

The split is generated using a grouped assignment procedure with random seed 42. The seed initializes the ordering of programming tasks, after which tasks are assigned as indivisible groups using a greedy procedure that seeks to preserve both the distribution of reference outcomes and the overall number of records across partitions while respecting the target number of tasks in each split. The split therefore combines seeded randomization with grouped stratification rather than assigning individual records independently.

Table 3. Task-level partition of the rationale-expanded dataset.

Split	Tasks	Records
Training	366	12,078
Validation	84	2,772
Test	113	3,729
Total	563	18,579

Grouping by programming task prevents different generations of the same specification from appearing across training and evaluation partitions. It also keeps all judge-generated rationales associated with a task in the same partition. This prevents a detector from being trained on one implementation or rationale associated with a programming problem and subsequently evaluated on another record derived from the same problem.

The complete rationale-expanded dataset is strongly imbalanced. *Correct* occurs in approximately 72.4% of the records, *Functional Error* in 21.3%, *Runtime Error* in 6.6%, and *Syntax Error* in 1.5%. Because *Runtime Error* and *Functional Error* may co-occur, these outcome frequencies are multilabel frequencies and are not required to sum to 100%.

The dominance of *Correct* motivates the class-balancing experiments. Without balancing, a detector may obtain comparatively strong aggregate performance by favoring the majority outcome rather than learning patterns associated with less frequent errors. Sampling for the balanced configurations is performed only after the task-level split has been created and affects only the training partition. Validation and test records remain unchanged, and balancing cannot move a task from one partition to another.

4.4.3 Fine-Tuning and Hyperparameter Search

Gemma 4 E2B, Gemma 4 E4B, and Gemma 3 4B are used as base models for specialized detector training. Each training example is formatted as a three-turn conversation. The system message defines the classification task and its four possible outcomes. The user message contains the programming-task description and generated code. The assistant target contains the execution-derived error outcomes and their corresponding judge-generated explanations.

Training uses QLoRA with 4-bit NF4 quantization (Dettmers et al., 2023), applying low-rank adapters following LoRA (Hu et al., 2022). Table 4 presents the hyperparameter search space.

The study evaluates training strategies with different optimization objectives and class distributions. Under the original distribution, the Accuracy-oriented configuration uses the standard causal language-modeling loss and selects the best checkpoint according to validation Accuracy. The Macro-F1-oriented configuration uses a label-weighted loss that assigns greater importance to the class-name tokens and

Table 4. Hyperparameters explored during specialized detector training.

Parameter	Values
LoRA rank (r)	{8, 16, 32}
LoRA alpha (α)	{8, 16, 32}
LoRA dropout	{0.05, 0.1}
Learning rate	{5e-5, 1e-4}
Bias	{none, all, lora_only}
Epochs	{3, 4}

selects the best checkpoint according to validation Macro-F1.

Additional configurations investigate class balancing within the training partition. The balancing procedure reduces the number of *Correct* examples by downsampling rather than duplicating minority examples. Sampling operates on the primary-label representation used by the dataset pipeline. A fully balanced *Runtime Error*:*Functional Error*:*Correct* ratio of 1:1:1 was explored during development, but it produced inferior validation behavior under both Accuracy-oriented and Macro-F1-oriented selection. The retained balanced configurations therefore use a 1:3:3 ratio, preserving more *Functional Error* and *Correct* examples while substantially reducing the dominance of the majority class.

Syntax Error examples are excluded from the balanced training configurations because they represent only a small fraction of the available training data and their observable outcome can be identified directly through deterministic compilation. Rather than duplicating the limited Syntax examples to force artificial balance, the balanced configurations focus on distinguishing *Runtime Error*, *Functional Error*, and *Correct*. Syntax examples remain unchanged in the validation and test partitions, so the effect of removing them from balanced training remains visible in the evaluation.

Hyperparameter search is performed with Optuna (Akiba et al., 2019) using the Tree-structured Parzen Estimator sampler with multivariate sampling and constant-liar. The search space covers the base model, LoRA rank and alpha, LoRA dropout, learning rate, number of epochs, and bias configuration. For each training strategy, the retained model is the configuration that maximizes its corresponding selection metric on the validation partition. The test partition is used only after model selection for the final evaluation.

4.5 Static Analysis Configuration

We use Python compilation and the Pyright static analyzer as deterministic sources of external evidence. Python compilation is associated with *Syntax Error*, while Pyright is used to identify static conditions associated with potential *Runtime Error*.

The experiments use Python 3.13.15 and Pyright 1.1.414. Python compilation uses the language's built-in compilation mechanism (Python Software Foundation, 2026) to parse candidate source code. A compilation failure produces a *Syntax Error* signal, whereas successful compilation indicates only that no syntax failure was detected. Compilation does not execute the generated program and is not used to infer *Runtime Error* or *Functional Error*.

Pyright (Microsoft, 2026) analyzes candidate source without executing it. It is configured using Python 3.13 semantics, its default type-checking mode, error-severity diagnostics, and structured output for parsing. Diagnostics are mapped to *Runtime Error* only when they belong to the runtime-related categories summarized in Table 5. Parsing diagnostics that indicate malformed source are instead mapped to *Syntax Error*. Other diagnostics outside the selected categories are ignored.

Table 5. Pyright diagnostic groups mapped to potential Runtime Errors.

Diagnostic group	Examples of detected problems
Name and binding errors	Undefined, unbound, or possibly unbound variables.
Call and argument errors	Invalid function calls, incompatible arguments, or invalid call signatures.
Type consistency errors	Incompatible assignments, return values, general type mismatches, or invalid type arguments.
Attribute, indexing, and operator errors	Invalid member access, unsupported indexing, or invalid operand combinations.
Optional-value misuse	Calling, indexing, iterating over, or accessing members of values that may be *None*.

Pyright therefore does not establish that a runtime failure will occur. Instead, it identifies static conditions associated with potential runtime failures that compilation alone cannot expose. Neither compilation nor Pyright is used as a detector of *Functional Error*, since functional correctness depends on whether the implementation satisfies the requested behavior. If a tool fails to execute or produces output that cannot be parsed, the failure is recorded and no error prediction is derived from that tool execution.

4.6 Tool-Assisted Experiments

Two experiments investigate the use of static-analysis evidence: error detection with static feedback and iterative code generation with static feedback. Figures 1 and 2 summarize the corresponding protocols.

4.6.1 Error Detection with Static Feedback

The tool-assisted detection experiment uses the same 113-task test partition employed in RQ2. For each sample, the model receives the programming-task description, the generated code, and the static feedback produced for that code by Python compilation and Pyright.

This experiment provides a controlled comparison with the general error-detection condition. For the same model and generated sample, the task description, code, classification prompt, output schema, temperature, and evaluation metrics remain unchanged. The additional static feedback is the only difference between the two detector inputs.

The feedback reports whether the source compiles and, when compilation succeeds, whether Pyright identifies a runtime-related diagnostic. Relevant diagnostic information is also supplied to the model. When neither mechanism reports an issue, the feedback explicitly states that no *Syntax Error* or statically detectable *Runtime Error* was identified and that this absence does not establish functional correctness.

The prompt further states that the tools provide incomplete evidence, that the absence of a diagnostic does not imply that the candidate is *Correct*, and that these mechanisms do not generally determine *Functional Error*. The model then predicts one or more outcomes among *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*.

Static feedback is treated only as supporting evidence and never replaces the execution-derived reference labels. Those labels are used only after inference to compute the evaluation metrics. As in the general detection experiment, an unparseable detector response is represented as an empty prediction set and remains in the evaluation.

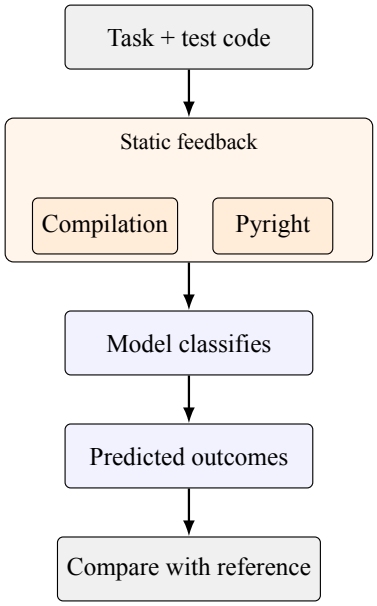


Figure 1. Tool-assisted error-detection protocol. Static analysis is supplied as supporting evidence before the model predicts the execution-based outcomes.

4.6.2 Iterative Code Generation with Static Feedback

The second tool-assisted experiment evaluates an iterative generation protocol in which static-analysis feedback is provided during code generation. The model first produces a candidate solution for the programming task. Each candidate is analyzed by Python compilation and, only when compilation succeeds, by Pyright. The tools therefore operate in short-circuit order: a candidate that fails compilation is not subsequently analyzed by Pyright.

The resulting diagnostics are returned to the model, which performs a self-review and decides whether to submit the current candidate or revise the implementation. The prompt contains the programming task, the current candidate, and the feedback generated for the current round. Previous rounds are not included in the prompt history.

When the tools report no issue, the feedback states that no syntax or statically detectable runtime problem was identified, while explicitly noting that this does not establish functional correctness. The model may therefore still revise the implementation even when the tools provide no diagnostic.

When a revision is produced, it completely replaces the previous candidate. No execution-based correctness test is performed before this substitution. If the response does not conform to the expected control structure but contains usable code, the direct-response execution treats that code as a revised candidate. When the response indicates submission, the current candidate is retained. Malformed responses are not removed from the experimental process.

The generation and review stages use a temperature of 0.0. The iterative process terminates when the model submits its current candidate or when five rounds have been completed. EvalPlus results are never supplied to the model during this process.

Within the iterative experiment, EvalPlus is executed only on the final candidate. This evaluation is distinct from the EvalPlus execution previously performed for the original code-generation experiment in RQ1. The original RQ1 outcome and the final outcome from the iterative protocol are subsequently compared for the same model and programming task to characterize whether the observed execution outcome improves, remains unchanged, or regresses.

This comparison evaluates two complete generation protocols rather than isolating the causal effect of static-analysis feedback. The original code-generation experiment uses a temperature of 1.0, whereas the iterative protocol uses a temperature of 0.0 and additionally introduces static feedback, self-review, and repeated revision. Results from this experiment are therefore interpreted as the behavior of the complete tool-assisted iterative protocol relative to the original generation condition.

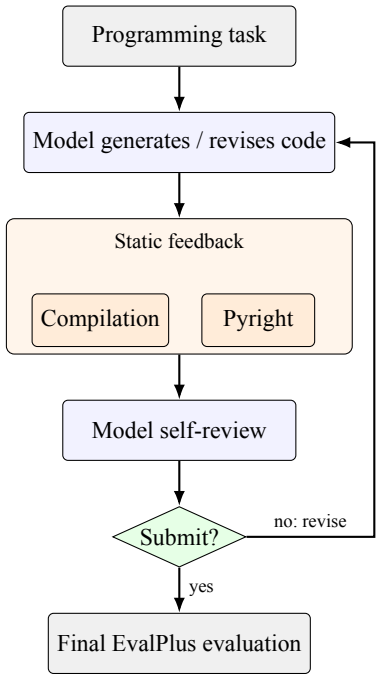


Figure 2. Tool-assisted iterative code-generation protocol. EvalPlus is applied to the final candidate and its result is not exposed to the model during generation or revision.

5 Results and Discussion

5.1 RQ1: To what extent can different language models generate correct code?

Figure 3 shows that *Correct* is the predominant outcome for every evaluated model, although the observed correctness rate varies considerably across configurations. The proportion of Correct solutions ranges from 62.2% to 80.8%. Muse Glimmer 30B achieves the highest observed result, solving 455 of the 563 tasks correctly (80.8%), followed by GPT-OSS 20B with 445 tasks (79.0%) and Gemma 4 31B with 443 tasks (78.7%). Gemma 3 4B presents the lowest correctness rate, with 350 correct solutions (62.2%). These results show that the evaluated SLMs are generally capable of producing correct solutions for a substantial portion of the benchmark, while also revealing meaningful differences among the evaluated configurations.

Model size alone does not explain all of these differences. A particularly relevant case appears within the Qwen 2.5 Coder family, where the 7B configuration solves 413 tasks correctly (73.4%), compared with 378 tasks (67.1%) for the 32B configuration. Since these models belong to the same family, the result shows that the larger variant does not necessarily outperform its smaller counterpart under the evaluated generation protocol. This observation should not be generalized into a broader inverse relationship between parameter count and correctness, since several of the strongest overall results are obtained by comparatively larger SLM configurations, including Muse Glimmer 30B, GPT-OSS 20B, and Gemma 4 31B. Instead, the Qwen result indicates that increasing the number of parameters within a model family does not by itself guarantee better performance on these function-level tasks.

The two substantially larger comparison models also fall within the performance range observed for the SLMs. DeepSeek V4.1 Flash and MiMo V2.5 each solve 413 tasks correctly (73.4%). This result is reported here to contextualize the observed correctness range, while the comparison between SLMs and substantially larger models is examined in greater detail in RQ7.

The distribution of errors provides additional insight into how code generation fails. *Functional Error* is the predominant failure outcome across the evaluated models, accounting for approximately 15% to 33% of their generations. *Runtime Error* is considerably less frequent, generally occurring in approximately 4% to 9% of the generated solutions. Thus, the primary difficulty is not producing Python code that immediately fails during execution, but producing executable implementations that fully satisfy the requested behavior. This distinction is particularly relevant to code hallucinations because a function may appear plausible and execute successfully while still deviating from the task specification.

Syntax Error is uncommon for most models, generally remaining below 3%, but some configurations exhibit model-specific output-format failures. Ornith 1.5 35B is the clearest case, with approximately 10% of its generations classified as Syntax Error. Manual inspection shows that 91.2% of Ornith's Syntax cases result from violations of the expected output structure rather than malformed Python syntax, partic-

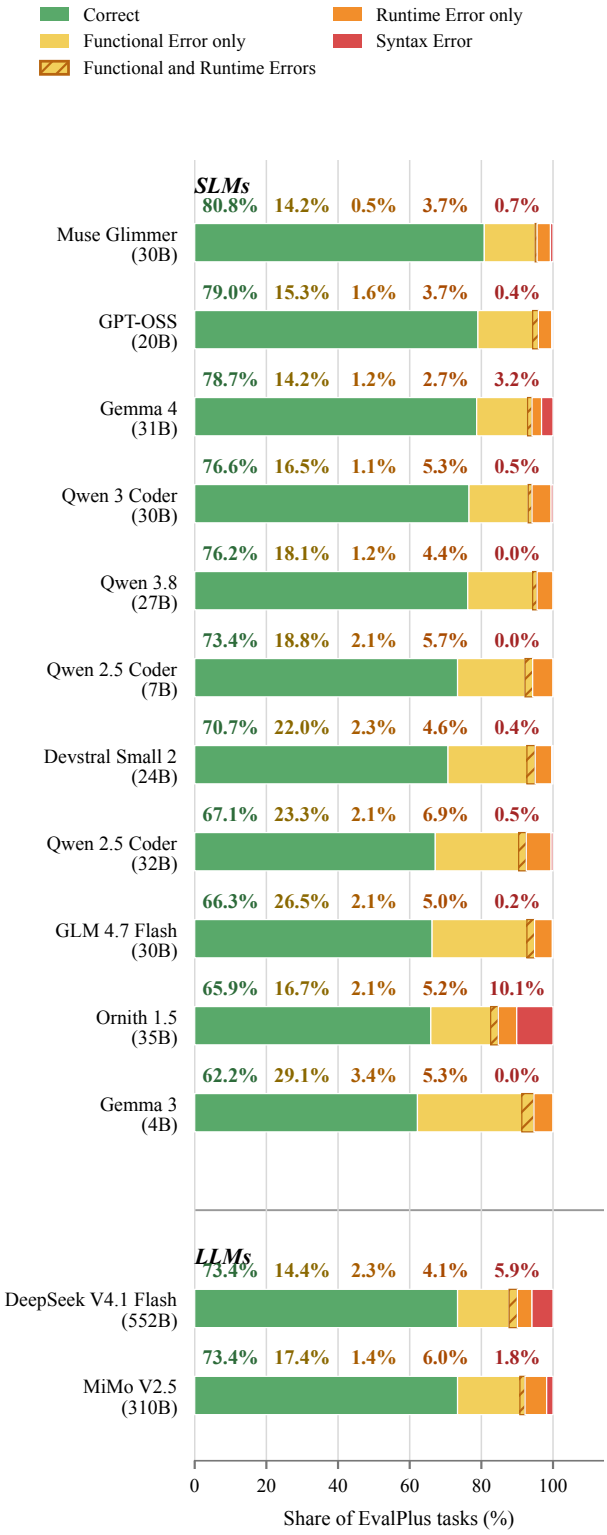


Figure 3. Code-generation outcomes across the 563 EvalPlus tasks. Each horizontal bar partitions the tasks into mutually exclusive outcomes; diagonal hatching marks programs labeled with both Functional and Runtime Errors. The five values above each bar report, respectively, Correct, Functional Error only, Functional and Runtime Errors, Runtime Error only, and Syntax Error.

Table 6. Error-detection performance by model (%). CR, FR, RR, and SR denote recall for Correct, Functional Error, Runtime Error, and Syntax Error, respectively. MF1(NS) denotes Macro-F1 computed without the Syntax Error class.

Model	Acc	MF1	CR	FR	RR	SR	MF1(NS)
General-purpose models							
Muse Glimmer 30B	73.1	53.8	83.3	55.5	27.0	52.9	61.3 (+7.5)
Gemma 4 31B	72.4	48.5	83.7	50.1	24.1	78.4	55.8 (+7.3)
Qwen 3.8 27B	62.6	42.6	62.9	75.9	12.7	64.7	48.6 (+5.9)
Qwen 2.5 Coder 7B	62.5	33.7	72.3	46.7	2.5	17.6	39.4 (+5.7)
Qwen 2.5 Coder 32B	60.8	41.0	71.1	38.1	11.4	76.5	43.3 (+2.3)
Qwen 3 Coder 30B	60.6	37.8	63.8	65.9	9.3	21.6	44.1 (+6.3)
Ornith 1.5 35B	56.3	35.7	55.4	78.1	8.0	35.3	43.1 (+7.4)
Devstral Small 2 24B	53.9	38.1	54.9	63.6	17.7	35.3	43.3 (+5.2)
GLM 4.7 Flash 30B	50.4	37.3	49.3	64.0	26.2	94.1	43.5 (+6.3)
Gemma 4 E2B IT	48.1	33.1	47.8	65.4	16.5	5.9	40.6 (+7.5)
Gemma 4 E4B IT	45.9	37.0	38.0	86.4	31.6	47.1	41.8 (+4.7)
Gemma 3 4B IT	38.5	26.7	30.0	82.0	10.1	5.9	32.4 (+5.7)
Gemma 3 4B	32.9	23.4	22.6	84.7	6.3	17.6	27.3 (+3.9)
GPT-OSS 20B	25.1	39.0	17.5	55.8	32.9	72.5	40.4 (+1.4)
Fine-tuned models							
Best Acc	77.8	50.9	85.3	69.1	27.8	11.8	63.1 (+12.2)
Weighted MF1	73.5	54.2	75.9	77.9	32.9	64.7	62.9 (+8.7)
Balanced Acc	67.8	38.1	74.4	65.8	15.2	0.0	50.9 (+12.7)
Balanced MF1	63.7	36.0	68.6	65.8	13.9	0.0	48.0 (+12.0)
Large models							
DeepSeek V4.1 Flash	75.5	51.0	89.3	51.6	14.8	35.3	54.1 (+3.1)
MiMo V2.5	51.5	39.7	47.2	79.7	21.5	96.1	44.7 (+5.0)

ularly when the model returns only a function body instead of the complete function definition required by the benchmark. More broadly, only 20% of the outcomes classified as Syntax Error correspond to genuine Python syntax errors, representing approximately 0.29% of all generated solutions. Consequently, the Syntax category captures not only failures in Python grammar but also failures to satisfy the expected code-generation interface, as defined by the operational classification procedure used in this study.

Overall, the evaluated models usually produce structurally executable Python code, while incorrect functional behavior remains the dominant source of failure. The strongest models generate correct solutions for close to four fifths of the benchmark, but even these configurations continue to produce implementations that execute without fully satisfying the task specification. The comparison within Qwen 2.5 Coder further shows that a larger configuration does not necessarily improve correctness within the same model family, although the highest overall correctness rates are still observed among several of the more capable SLM configurations.

5.2 RQ2: To what extent can SLMs correctly identify errors in generated code?

Table 6 summarizes the error-detection performance of the evaluated models. Accuracy measures whether the complete predicted label set matches the execution-derived reference, while Macro-F1 gives equal importance to the four outcome categories. CR, FR, RR, and SR denote recall for *Correct*, *Functional Error*, *Runtime Error*, and *Syntax Error*, respectively. MF1(NS) reports Macro-F1 after excluding *Syntax Error* as a sensitivity analysis. Although the table also reports the fine-tuned and substantially larger models to avoid repeating the same results across multiple tables, this RQ discusses only the general-purpose SLMs.

Among the general-purpose SLMs, Muse Glimmer 30B presents the strongest aggregate detection results, reaching 73.1% Accuracy and 53.8% Macro-F1. Gemma 4 31B follows closely in Accuracy at 72.4%, with a Macro-F1 of 48.5%. The remaining models exhibit substantially different combinations of aggregate and class-specific performance. This difference between Accuracy and Macro-F1 is important because the task is strongly imbalanced: a model can obtain comparatively high Accuracy by recognizing the more frequent outcomes while remaining weak on less frequent error categories.

The two Qwen 2.5 Coder configurations illustrate this distinction. The 7B model obtains slightly higher Accuracy than the 32B model, 62.5% compared with 60.8%, but the 32B model reaches a substantially higher Macro-F1, 41.0% compared with 33.7%. The difference is primarily associated with the minority error categories: Runtime Recall increases from 2.5% to 11.4%, while Syntax Recall increases from 17.6% to 76.5%. Therefore, Accuracy alone would obscure an important difference in how these two configurations distribute their predictions across error types.

Detection capability is strongly dependent on the error category. *Functional Error* is recognized comparatively well by

several models. Gemma 4 E4B IT reaches 86.4% Functional Recall, Gemma 3 4B reaches 84.7%, Gemma 3 4B IT reaches 82.0%, Ornith 1.5 35B reaches 78.1%, and Qwen 3.8 27B reaches 75.9%. However, strong Functional Recall does not imply strong overall detection. Gemma 3 4B, for example, reaches 84.7% Functional Recall but only 32.9% Accuracy, together with 22.6% Correct Recall and 6.3% Runtime Recall. The model therefore recognizes many functional failures while struggling to distinguish the other outcomes.

Runtime Error is the most difficult category for the general-purpose SLMs. GPT-OSS 20B obtains the highest Runtime Recall at 32.9%, followed by Gemma 4 E4B IT at 31.6%, Muse Glimmer 30B at 27.0%, GLM 4.7 Flash at 26.2%, and Gemma 4 31B at 24.1%. No general-purpose SLM identifies more than approximately one third of the Runtime Errors. This contrasts with Functional Error, for which several models detect more than three quarters of the reference cases. Runtime failures therefore remain a substantial limitation of model-based error detection under this protocol.

Syntax Error presents a different pattern, with substantial variation across models. GLM 4.7 Flash reaches 94.1% Syntax Recall, followed by Gemma 4 31B at 78.4%, Qwen 2.5 Coder 32B at 76.5%, and GPT-OSS 20B at 72.5%. At the opposite end, Gemma 4 E2B IT and Gemma 3 4B IT each reach only 5.9%. Thus, unlike Runtime Error, where performance is consistently low, the ability to recognize Syntax Error is highly model-dependent.

GPT-OSS provides one of the clearest examples of the separation between code generation and error detection. In RQ1, it generates a Correct solution for 79.0% of the tasks, placing it among the strongest generators. In the detection experiment, however, it reaches only 25.1% Accuracy and 17.5% Correct Recall. At the same time, it obtains the highest Runtime Recall among the general-purpose SLMs and 72.5% Syntax Recall. Its low aggregate Accuracy therefore does not reflect uniformly poor error recognition, but rather a strongly asymmetric prediction profile in which the model is particularly weak at recognizing code that is actually Correct.

Because *Syntax Error* is rare and can also be identified directly through deterministic compilation, Table 6 additionally reports Macro-F1 without this category. This ablation evaluates how strongly the rare Syntax class affects the aggregate macro metric. Muse Glimmer increases from 53.8% to 61.3%, Gemma 4 31B from 48.5% to 55.8%, Qwen 3.8 from 42.6% to 48.6%, and Qwen 2.5 Coder 7B from 33.7% to 39.4%. In contrast, GPT-OSS changes only from 39.0% to 40.4% because Syntax is already one of its stronger categories. Since Macro-F1 assigns equal importance to each class, the ablation shows that performance on the uncommon Syntax category can noticeably affect the aggregate score even though Syntax represents only a small fraction of the dataset. MF1(NS) is therefore used only as a sensitivity analysis, while the primary Macro-F1 continues to include all four outcome categories.

Beyond detecting errors in code generated by other models, RQ2 also examines whether a model can recognize errors in its own outputs. Table 7 reports this self-error detection analysis. Self-Det. is the proportion of erroneous solutions generated by a model for which its subsequent prediction exactly matches the execution-derived reference set. The class-specific columns report detection within the model's own Functional, Runtime, and Syntax error subsets. The numerator and denominator are reported because each model produces a different number and distribution of errors. ND indicates that the model produced no reference error of that category in the evaluated test split.

Table 7. Self-error detection by generator model. Self-Det. reports exact-set detection over the model's own erroneous solutions. Class-specific cells report detected/reference counts. ND indicates that the model produced no reference error of that category.

Model	Self-Det.	Funct.	Runt.	Synt.
Qwen 3.8 27B	67.9% (55/81)	75.4% (52/69)	25.0% (3/12)	ND
Gemma 3 4B	60.4% (87/144)	77.3% (102/132)	0.0% (0/30)	ND
GLM 4.7 Flash 30B	55.0% (61/111)	65.6% (63/96)	20.0% (3/15)	100.0% (3/3)
Qwen 2.5 Coder 7B	54.4% (49/90)	66.7% (46/69)	14.3% (3/21)	ND
Ornith 1.5 35B	53.8% (63/117)	80.0% (48/60)	16.7% (4/24)	38.5% (15/39)
Devstral Small 2 24B	52.3% (58/111)	64.4% (58/90)	9.1% (3/33)	ND
Qwen 3 Coder 30B	42.3% (33/78)	51.5% (34/66)	0.0% (0/15)	ND
Qwen 2.5 Coder 32B	33.3% (37/111)	36.9% (31/84)	19.4% (7/36)	ND
Muse Glimmer 30B	19.0% (12/63)	25.9% (14/54)	25.0% (3/12)	ND
Gemma 4 31B	13.6% (9/66)	6.7% (3/45)	16.7% (3/18)	33.3% (3/9)
GPT-OSS 20B	5.3% (3/57)	15.7% (8/51)	0.0% (0/21)	ND

Self-error detection varies substantially across the evaluated SLMs. Qwen 3.8 obtains the highest exact-set Self-Detection rate at 67.9% (55/81), followed by Gemma 3 4B at 60.4% (87/144). At the opposite end, GPT-OSS detects the complete error set for only 5.3% (3/57) of its own erroneous solutions. This creates a particularly important contrast with RQ1: GPT-OSS is one of the strongest code generators, with 79.0% Correct generation, but is the weakest model in exact-set self-error detection. Strong generation capability therefore does not imply that a model can reliably recognize the errors that remain in its own outputs.

The class-specific self-evaluation results follow a pattern similar to general detection. Functional Errors are comparatively easier to recognize, with Ornith reaching 80.0%, Gemma 3 4B reaching 77.3%, and Qwen 3.8 reaching 75.4%. Runtime self-detection remains low: Qwen 3.8 and Muse Glimmer each detect 3 of 12 Runtime cases, GLM detects 3 of 15, and Qwen 2.5 Coder 32B detects 7 of 36, while some models detect none. These percentages must be interpreted together with their denominators because the number of errors produced by each model differs substantially. GLM, for example, recognizes all of its own Syntax Errors, but this corresponds to only three cases.

Overall, the evaluated SLMs exhibit heterogeneous and strongly class-dependent error-detection capabilities. Func-

tional Errors can be recognized with relatively high recall by several models, whereas Runtime Errors remain difficult across the evaluated configurations. Syntax detection varies substantially between models and can noticeably affect Macro-F1 despite the small size of the class. The self-evaluation results further show that code-generation ability and error-recognition ability are distinct: a model that frequently generates correct code may still be unable to reliably identify the errors present in the smaller set of solutions that it gets wrong.

5.3 RQ3: To what extent can specialized SLMs improve the detection of errors in generated code?

The four specialized detectors are compared with their Gemma 4 E4B base model. Table 8 summarizes Accuracy and Macro-F1 under the paired comparison used for this analysis. Values in parentheses report the difference in percentage points relative to the base model. The paired values can differ slightly from the standalone results in Table 6 because this comparison uses only rows available for both the base and specialized conditions.

Table 8. Performance of the specialized detectors relative to the Gemma 4 E4B base model. The base reaches 45.9% Accuracy and 37.1% Macro-F1 on the paired subset. Values in parentheses indicate the difference in percentage points. Green cells indicate improvements whose paired bootstrap interval excludes zero, while gray cells indicate differences whose interval includes zero.

Specialized Model	Accuracy	Macro-F1
Best Acc	77.9 (+32.0)	51.0 (+13.9)
Weighted MF1	73.6 (+27.7)	54.3 (+17.2)
Balanced Acc	67.9 (+22.0)	38.1 (+1.0)
Balanced MF1	63.8 (+17.9)	36.0 (-1.1)

All four specialized configurations increase Accuracy relative to the base model, but their effects on Macro-F1 differ substantially. Best Acc produces the largest Accuracy improvement, increasing the metric by 32.0 percentage points, with a paired task-level bootstrap interval of [25.0, 39.2]. Its Macro-F1 also increases by 13.9 points, with an interval of [6.7, 21.0]. Weighted MF1 produces a slightly smaller Accuracy gain of 27.7 points [21.3, 34.2], but the largest Macro-F1 improvement, increasing it by 17.2 points [11.9, 22.3]. These results show that specialization improves detection under both optimization objectives, while the objective determines which aspect of performance benefits most strongly.

The Accuracy-oriented detector achieves its improvement primarily by becoming substantially more likely to predict *Correct*. Figure 4 shows that, for the base model, only 38.3% of samples whose primary reference label is Correct are predicted as Correct, while 49.2% are predicted as Functional. After Accuracy-oriented specialization, Correct-to-Functional confusion decreases to 13.1%, while Correct predictions increase substantially. However, this shift also affects the error classes. Functional-to-Correct increases from 4.5% to 25.2%, Runtime-to-Correct from 15.8% to 34.2%, and Syntax-to-Correct from 6.2% to 37.5%. Thus, the Accu-

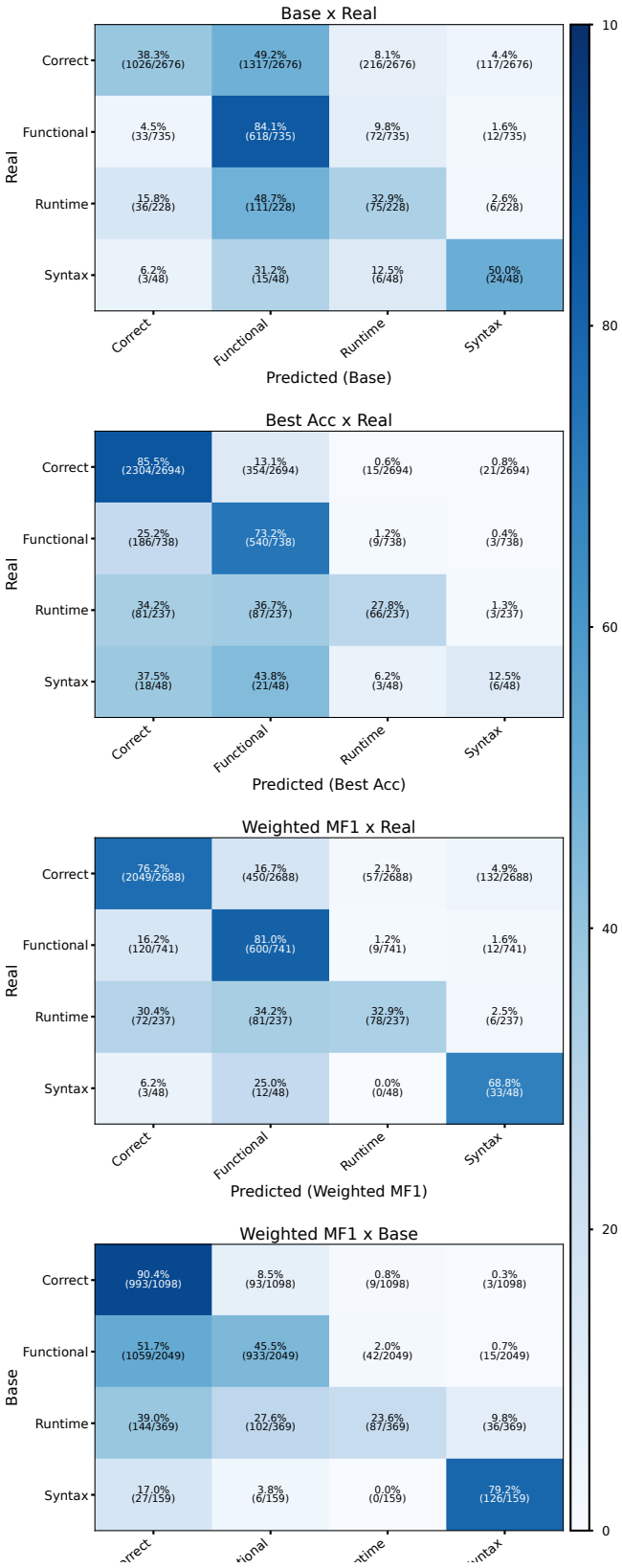


Figure 4. Row-normalized confusion matrices under a primary-label reduction with priority Syntax, Runtime, Functional, and Correct. Each cell reports the proportion and the counts as predicted/reference total. The first three matrices compare the base, Best Acc, and Weighted MF1 predictions with the execution-derived reference. The last matrix compares Weighted MF1 predictions directly with those of the base model. Unparseable outputs are excluded only from this primary-label visualization.

racy gain is accompanied by a stronger tendency to predict the dominant Correct outcome.

The Macro-F1-oriented detector exhibits a different behavior. In the full multilabel evaluation reported in Table 6, Best Acc reaches 77.8% Accuracy and 50.9% Macro-F1, whereas Weighted MF1 reaches 73.5% Accuracy and 54.2% Macro-F1. The difference is also visible at the class level. Best Acc obtains 85.3% Correct Recall, 69.1% Functional Recall, 27.8% Runtime Recall, and 11.8% Syntax Recall. Weighted MF1 reduces Correct Recall to 75.9%, but increases Functional Recall to 77.9%, Runtime Recall to 32.9%, and Syntax Recall to 64.7%. The Macro-F1-oriented objective therefore sacrifices part of the strong preference for Correct in exchange for a more balanced error-detection profile.

The confusion matrices reinforce this difference. Relative to Best Acc, Weighted MF1 reduces Functional-to-Correct confusion from 25.2% to 16.2% and Syntax-to-Correct confusion from 37.5% to 6.2%. The improvement is especially pronounced for *Syntax Error*, whose recall increases from 11.8% under Best Acc to 64.7% under Weighted MF1. This result shows that the training objective changes not only the aggregate score but also which types of errors the specialized detector is able to recognize.

Runtime Error remains the most difficult category after specialization. Weighted MF1 reaches 32.9% Runtime Recall, which is higher than Best Acc at 27.8%, but still considerably below its recall for Functional and Syntax Errors. The primary-label matrices similarly show substantial confusion between Runtime, Functional, and Correct predictions. Specialization therefore changes how Runtime cases are distributed among predictions, but does not eliminate the difficulty of identifying this error category.

The explicitly balanced training strategies produce a different result. Balanced Acc increases Accuracy by 22.0 percentage points relative to the base, with a paired bootstrap interval of [15.0, 29.1], while Balanced MF1 increases it by 17.9 points [11.4, 24.5]. Their Macro-F1 changes, however, are much smaller. Balanced Acc changes Macro-F1 by +1.0 point, with an interval of [-4.6, 6.6], while Balanced MF1 changes it by -1.1 points, with an interval of [-6.5, 4.4]. The intervals therefore include zero for both balanced configurations, and the observed differences do not provide clear evidence of a Macro-F1 improvement over the base model.

The class-level results help explain this behavior. Balanced Acc reaches 74.4% Correct Recall and 65.8% Functional Recall, but only 15.2% Runtime Recall. Balanced MF1 reaches 68.6% Correct Recall, 65.8% Functional Recall, and 13.9% Runtime Recall. Both configurations obtain 0% Syntax Recall. As described in the experimental setup, Syntax examples were excluded from balanced training because of their low representation and the availability of deterministic compilation for this outcome. The unchanged validation and test partitions preserve these cases during evaluation, making the loss of Syntax detection directly observable.

The final matrix in Figure 4 further illustrates that specialization changes prediction behavior rather than simply correcting every previous decision. Among samples predicted as Correct by the base model, 90.4% remain Correct under Weighted MF1. In contrast, only 23.6% of samples previously predicted as Runtime remain Runtime, while 51.7% of samples previously predicted as Functional are reclassified as Correct. These transitions do not indicate correctness by themselves because the matrix compares two sets of predictions rather than predictions against the reference. Instead, they illustrate how strongly fine-tuning changes the detector's decision boundaries.

Overall, specialization improves error detection, but the benefits depend on the optimization objective. Accuracy-oriented training produces the largest gain in exact-set Accuracy and shifts the detector toward recognizing *Correct*, whereas Macro-F1-oriented training yields the largest improvement in Macro-F1 and a more balanced profile across the error categories. The evaluated balancing strategies also increase Accuracy relative to the base model, but do not produce a clear Macro-F1 improvement. Across all specialized configurations, *Runtime Error* remains the most difficult outcome, indicating that specialization improves the detector substantially without eliminating the main class-specific limitation observed in the general-purpose models.

5.4 RQ4: How do static-analysis techniques compare with SLMs in detecting errors in generated code?

This RQ compares the standalone detection capability of Python compilation and Pyright with the corresponding class-specific results of the language models. The comparison is restricted to the outcomes directly targeted by each technique: Python compilation for *Syntax Error* and Pyright for statically detectable *Runtime Error*. The static techniques do not produce predictions for *Functional Error* or *Correct*, so the comparison focuses on the error categories for which a direct correspondence can be established.

Table 9. Standalone static-tool detection on the held-out test split and on the complete generated set. Values report detected/reference cases and recall.

Tool	Target	Test	Global
Python compile	Syntax	17/17 (100.0%)	90/90 (100.0%)
Pyright	Runtime	7/79 (8.9%)	45/408 (11.0%)

Python compilation provides a clear advantage for *Syntax Error*. It detects all 17 held-out Syntax cases and all 90 cases in the complete generated set. In contrast, only 6 of the 14 general-purpose SLM configurations identify at least half of the Syntax Errors, and only three exceed 75% Syntax Recall. The median Syntax Recall among these models is approximately 41.2%. Although some individual SLMs exhibit strong syntactic detection, most do not approach the reliability of direct compilation. These results indicate that SLMs have limited and inconsistent standalone capability for verifying syntactic validity when compared with a deterministic mechanism designed to check this property.

The relationship is different for *Runtime Error*. Pyright detects only 7 of the 79 held-out Runtime cases, corresponding to 8.9% recall, and 45 of 408 cases globally, corresponding to 11.0%. Eleven of the 14 general-purpose SLM configurations exceed Pyright's held-out Runtime Recall, with a median of approximately 14.6%. GPT-OSS reaches the highest

observed Runtime Recall at 32.9%. Runtime detection remains difficult for the language models themselves, but most evaluated SLMs identify a larger fraction of these failures than the selected Pyright diagnostic mapping. This indicates that model-based reasoning can recognize execution-related problems that are not exposed by the static conditions captured in the evaluated Pyright configuration.

The comparison therefore reveals a clear difference in the strengths of the two approaches. SLMs do not provide the same reliability as deterministic compilation for detecting syntactic failures, whereas they can provide greater coverage of Runtime failures than the static-analysis configuration evaluated in this study. Static analysis is particularly effective when the target property can be checked directly, while SLMs can contribute additional detection capability for failures that require broader reasoning about program behavior. This division of strengths motivates RQ5, which investigates whether language models can improve their own error detection when the static evidence is supplied explicitly.

5.5 RQ5: How does static-analysis feedback affect the ability of language models to detect errors in generated code?

This RQ evaluates how error-detection behavior changes when the same generated code used in RQ2 is accompanied by static-analysis feedback. The task, candidate code, classification prompt, temperature, output schema, and execution-derived reference remain unchanged, with the additional tool evidence being the only difference between the detector inputs. Table 10 reports the resulting metrics and their changes relative to RQ2.

The clearest effect of static-analysis feedback is observed for *Syntax Error*. Eighteen of the 20 evaluated configurations reach at least 94% Syntax Recall after receiving the tool output. The only exceptions are the two balanced specialized detectors, both trained without Syntax examples. For many models, the change is substantial: Gemma 3 4B and Qwen 2.5 Coder 7B increase from 17.6% to 100.0%, Qwen 3 Coder 30B from 21.6% to 100.0%, and Gemma 4 E2B IT and Gemma 3 4B IT increase by approximately 88 percentage points. Across all evaluated configurations, the average increase in Syntax Recall is approximately 51.3 percentage points. Thus, when deterministic evidence is directly aligned with the target outcome, most models are able to incorporate it effectively.

The two balanced detectors provide an important exception to this pattern. Both were trained without Syntax examples, yet they react very differently to the same explicit compilation evidence. Balanced Acc remains at 0.0% Syntax Recall, whereas Balanced MF1 increases from 0.0% to approximately 81%. In contrast, Best Acc and Weighted MF1 both reach 100.0%. The availability of an explicit signal is therefore not sufficient by itself to determine the resulting prediction. The behavior acquired during specialization, including the training objective and exposure to the target class, influences how the model maps external evidence to its output labels.

The behavior for *Runtime Error* is notably different. Runtime Recall decreases in 17 of the 20 evaluated configurations, increases in only two, and remains unchanged in one. Across the evaluated conditions, the average change is approximately -5.4 percentage points. This occurs despite Pyright providing information specifically intended to support Runtime detection. A plausible interpretation is related to the limited coverage identified in RQ4: Pyright detects only 8.9% of the held-out Runtime failures. Consequently, most true Runtime cases are accompanied by feedback in which Pyright reports no mapped runtime-related diagnostic. Although the prompt explicitly states that the absence of a diagnostic does not establish the absence of an error, this negative tool signal may still influence the model away from predicting *Runtime Error*. The experiment does not isolate this mechanism directly, but the observed pattern is consistent with models assigning excessive weight to incomplete static evidence.

This contrast is visible in individual models. GLM 4.7 Flash loses 17.3 percentage points of Runtime Recall, Weighted MF1 loses 12.7, MiMo V2.5 loses 12.5, Gemma 4 E4B IT loses 11.4, and Best Acc loses 8.9. Only Gemma 3 4B and Qwen 2.5 Coder 7B show clear increases. Static evidence therefore behaves differently depending on its coverage: compilation provides a nearly complete signal for Syntax and is followed by a large improvement, whereas the sparse Pyright signal does not produce the same effect for Runtime and frequently coincides with worse Runtime detection.

Static feedback also systematically affects predictions for classes that neither tool directly determines. Correct Recall decreases in 12 of the 20 configurations, while Functional Recall increases in 15. The average changes are approximately -5.3 percentage points for Correct Recall and +6.8 points for Functional Recall. This pattern is consistent with a redistribution in which models become more likely to treat a candidate as functionally incorrect after receiving static-analysis information, even though compilation and Pyright provide no direct evidence about whether the implementation satisfies the task specification.

Qwen 2.5 Coder 7B provides the clearest example of this behavior. Its Correct Recall decreases by 48.8 percentage points while Functional Recall increases by 42.6 points. At the same time, Runtime Recall increases by 5.1 points and Syntax Recall increases by 82.4 points. Its Exact-set Accuracy consequently decreases by approximately 26 points despite large improvements in several error categories. Qwen 2.5 Coder 32B and Qwen 3 Coder show similar, although less extreme, shifts from Correct toward Functional predictions. The decrease in Accuracy in these cases therefore does not represent a uniform loss of detection capability, but a substantial change in the model's decision behavior across classes.

Macro-F1 also increases in 19 of the 20 evaluated configurations, with an average increase of approximately 13.6 percentage points. This result should not be interpreted as a uniform improvement across the error taxonomy. A major common contributor is the large increase in Syntax Recall, which rises by more than 50 percentage points on average and receives equal weight in the macro metric despite being a rare class. Runtime Recall, in contrast, decreases for most models, while Correct and Functional predictions frequently

Table 10. Detection results with static-analysis feedback, compared with the no-feedback setting of RQ2. Values are percentages and the value in parentheses is the change relative to RQ2.

Model	Acc	MF1	CR	FR	RR	SR
General-purpose models						
Muse Glimmer 30B	79 (+6)	71 (+17)	89 (+6)	58 (+3)	25 (-2)	100 (+47)
Gemma 4 31B	77 (+4)	66 (+18)	87 (+3)	56 (+6)	23 (-1)	100 (+22)
Qwen 3.8 27B	69 (+7)	56 (+13)	73 (+10)	69 (-6)	9 (-4)	100 (+35)
Qwen 2.5 Coder 7B	37 (-26)	47 (+13)	24 (-49)	89 (+43)	8 (+5)	100 (+82)
Qwen 2.5 Coder 32B	43 (-18)	50 (+9)	44 (-27)	47 (+9)	9 (-3)	100 (+24)
Qwen 3 Coder 30B	52 (-8)	46 (+9)	46 (-18)	86 (+20)	4 (-5)	100 (+78)
Ornith 1.5 35B	59 (+2)	52 (+16)	55 (-1)	84 (+6)	8 (+0)	100 (+62)
Devstral Small 2 24B	56 (+2)	53 (+15)	54 (-1)	71 (+7)	11 (-6)	100 (+65)
GLM 4.7 Flash 30B	46 (-4)	45 (+7)	37 (-13)	88 (+24)	9 (-17)	100 (+6)
Gemma 4 E2B IT	50 (+2)	49 (+16)	48 (+0)	68 (+3)	10 (-6)	94 (+88)
Gemma 4 E4B IT	46 (+0)	52 (+15)	35 (-3)	93 (+6)	20 (-11)	100 (+50)
Gemma 3 4B IT	36 (-2)	46 (+19)	25 (-5)	83 (+1)	8 (-3)	94 (+88)
Gemma 3 4B	36 (+3)	47 (+24)	24 (+2)	82 (-3)	9 (+3)	100 (+82)
GPT-OSS 20B	31 (+6)	50 (+11)	26 (+8)	52 (-4)	27 (-6)	100 (+27)
Fine-tuned models						
Best Acc	77 (+0)	68 (+17)	85 (+0)	65 (-4)	19 (-9)	100 (+88)
Weighted MF1	76 (+2)	68 (+14)	79 (+4)	75 (-3)	20 (-13)	100 (+31)
Balanced Acc	67 (-1)	36 (-2)	71 (-3)	71 (+5)	10 (-5)	0 (+0)
Balanced MF1	61 (-3)	56 (+20)	62 (-7)	74 (+8)	7 (-7)	81 (+81)
Large models						
DeepSeek V4.1 Flash	76 (+1)	64 (+13)	88 (-2)	55 (+4)	10 (-5)	100 (+65)
MiMo V2.5	47 (-4)	47 (+8)	37 (-10)	90 (+10)	9 (-13)	100 (+4)

shift in opposite directions. Exact-set Accuracy consequently presents a much less uniform pattern, increasing in 10 configurations, decreasing in eight, and remaining approximately unchanged in two.

GLM 4.7 Flash illustrates why the Macro-F1 increase cannot be explained only by improved Syntax detection. The model already reaches 94.1% Syntax Recall without feedback, so the tool raises this value by only approximately six percentage points. Nevertheless, its Macro-F1 increases by seven points. The principal change instead occurs in Functional Recall, which increases by approximately 24 points, while Correct Recall decreases by 13 points and Runtime Recall by 17 points. Thus, even when the direct syntactic contribution is small, static feedback can substantially alter decisions about classes for which the tools provide no direct information.

Overall, static-analysis feedback changes model behavior much more broadly than the classes directly targeted by the tools. Compilation provides a strong and consistently useful Syntax signal, while the incomplete Runtime evidence from Pyright frequently coincides with reduced Runtime Recall. At the same time, many models shift predictions from *Correct* toward *Functional Error*, demonstrating that tool output can influence judgments about properties that the tools do not evaluate. The contrasting behavior of the balanced specialized models further shows that the ability to exploit external evidence depends on the detector's learned decision behavior. Static feedback therefore does not act as a simple additional source of correct labels: its effect depends both on the coverage of the underlying tool and on how each model transforms that evidence into a classification.

5.6 RQ6: How does an iterative static-analysis feedback cycle affect the ability of language models to generate correct code?

This RQ compares the final code produced by the iterative tool-assisted protocol with the original generation results of RQ1. The comparison concerns two complete generation protocols. RQ1 uses a temperature of 1.0 and a single generation, whereas the iterative protocol uses a temperature of 0.0 together with static-analysis feedback, self-review, and up to five revision cycles. Consequently, the observed differences cannot be attributed exclusively to static analysis and are interpreted as changes under the complete iterative protocol.

The effect of the iterative protocol on final correctness is heterogeneous among the SLMs. Six of the eleven evaluated SLMs increase their proportion of *Correct* solutions relative to RQ1, while five decrease it. The largest positive change is observed for Qwen 2.5 Coder 32B, which increases Correct by approximately 8 percentage points, followed by Ornith 1.5 with approximately 6 points and Gemma 4 31B with approximately 3 points. In contrast, Qwen 2.5 Coder 7B, GLM 4.7 Flash, Qwen 3 Coder, Muse Glimmer, and Gemma 3 finish the iterative protocol with fewer Correct solutions than under the original generation condition. Static-analysis-assisted revision therefore does not produce a uniform improvement in final code correctness across SLMs.

Table 11. Code generation under the iterative static-analysis protocol, compared with RQ1. Values are percentages and the second line in each outcome cell reports the change relative to RQ1. Green changes indicate an improvement in the corresponding outcome, while red changes indicate a deterioration. Avg Cyc is the mean number of cycles.

Model	Avg Cyc	Correct	Funct.	Runtime	Syntax
Small language models					
Gemma 4 (31B)	1.29	82 +3	15 0	4 0	0 -3
GPT-OSS (20B)	1.46	80 +1	15 -2	4 -1	2 +1
Muse Glimmer (30B)	1.41	79 -2	16 +1	4 0	2 +2
Qwen 3.8 (27B)	1.17	78 +2	19 -1	4 -1	0 0
Qwen 2.5 Coder (32B)	1.36	75 +8	20 -5	7 -2	0 0
Qwen 3 Coder (30B)	1.57	74 -2	21 +4	6 -1	0 0
Devstral Small 2 (24B)	1.37	73 +2	22 -2	6 -1	0 0
Ornith 1.5 (35B)	1.68	72 +6	19 +1	6 -1	5 -6
Qwen 2.5 Coder (7B)	2.06	71 -3	24 +3	8 0	0 0
GLM 4.7 Flash (30B)	2.14	63 -3	25 -4	5 -2	8 +8
Gemma 3 (4B)	1.43	61 -2	34 +1	8 -1	0 0
Large language models					
DeepSeek V4.1 Flash (552B)	1.14	81 +7	16 -1	5 -1	0 -6
MiMo V2.5 (310B)	1.15	80 +7	16 -2	5 -2	0 -2

The error distribution shows that different models reach their final Correct proportion through different changes in failure type. Qwen 2.5 Coder 32B provides the clearest positive example. Its increase of approximately 8 points in Correct is accompanied by reductions of approximately 5 points in *Functional Error* and 2 points in *Runtime Error*, while Syntax remains near zero. The largest SLM improvement therefore cannot be explained only by correcting properties directly checked by compilation or Pyright. The iterative protocol can also change Functional outcomes even though the static tools provide no direct functional-correctness signal.

Ornith 1.5 presents a different improvement profile. Its Correct proportion increases by approximately 6 points, while Syntax decreases by approximately 6 points and Runtime decreases slightly. Functional Error does not improve and instead increases slightly. Gemma 4 31B shows an even more direct relationship between structural failures and final correctness: Correct increases by approximately 3 points while Syntax decreases by approximately the same amount, with comparatively small changes in the remaining outcomes. Thus, similar gains in Correct may originate from different changes in the underlying error distribution.

The GLM 4.7 Flash result illustrates the opposite phenomenon. Functional and Runtime failures decrease by approximately 4 and 2 percentage points, respectively, yet Syntax increases by approximately 8 points and the final proportion of Correct solutions decreases by approximately 3 points. Improving some observable failure categories therefore does not guarantee a better final result if the protocol simultaneously introduces another type of failure. This case is particularly notable because GLM reaches 94.1% Syntax Recall as a detector in RQ2, but produces substantially more Syntax failures under the iterative generation protocol. Recognizing a syntactic problem during classification and avoiding or repairing the same type of problem during generation are therefore distinct capabilities.

Runtime Error presents an important contrast with RQ5. When static feedback is supplied for classification, Runtime Recall decreases for most detector configurations. During iterative generation, however, the incidence of Runtime failures decreases for eight of the eleven SLMs and remains approximately unchanged for the other three in the rounded results reported in Table 11. The two substantially larger models also reduce Runtime failures. Qwen 2.5 Coder 32B presents the largest SLM reduction, followed by smaller decreases for GLM 4.7 Flash, Qwen 3.8, Ornith 1.5, GPT-OSS, Devstral Small 2, Qwen 3 Coder, and Gemma 3. This difference suggests that translating partial static evidence into an explicit *Runtime Error* label and using similar evidence as guidance for code revision are different tasks.

The protocol also affects Functional outcomes in both directions. Qwen 2.5 Coder 32B, GLM 4.7 Flash, GPT-OSS, Qwen 3.8, and Devstral Small 2 reduce Functional failures, while Qwen 3 Coder, Qwen 2.5 Coder 7B, Muse Glimmer, Ornith 1.5, and Gemma 3 increase them. This behavior is important because neither compilation nor Pyright directly evaluates whether the candidate satisfies the functional specification. Functional changes can therefore emerge from the model's self-review and revision behavior even when the external evidence concerns only syntactic or statically de-

tectable runtime properties.

The average number of cycles does not show a simple relationship with the final outcome. Although the protocol permits up to five rounds, the mean number of cycles ranges only from 1.14 to 2.14 across the evaluated models. Qwen 2.5 Coder 32B obtains the largest SLM increase in Correct with an average of only 1.36 cycles, whereas GLM 4.7 Flash averages 2.14 cycles and finishes with fewer Correct solutions. Qwen 2.5 Coder 7B similarly averages 2.06 cycles while decreasing Correct. Additional revision opportunities therefore do not by themselves imply better final code; the result depends on how effectively each model uses the available feedback during revision.

For the two hosted models, paired observations are available for both the original and iterative protocols. DeepSeek changes from 13 tasks that are Correct only under RQ1 to 54 tasks that are Correct only under the iterative protocol, while MiMo changes from 11 RQ1-only Correct tasks to 48 iterative-only Correct tasks. Exact two-sided McNemar tests yield $p < .001$ for both models. The retained SLM comparisons remain descriptive because equivalent paired raw observations are not available for those conditions.

Overall, the iterative protocol improves final correctness for some models but produces regressions for others. Its effects are also not restricted to the properties directly inspected by the static tools: some of the largest gains and losses involve Functional Error, while Runtime failures decrease more consistently during revision than Runtime Recall improves during classification in RQ5. The results therefore reinforce the distinction between error detection, interpretation of external evidence, and code revision. Because RQ1 and RQ6 also differ in decoding temperature, self-review, and the opportunity for repeated revision, these results characterize the complete iterative protocol rather than the isolated causal contribution of static-analysis feedback.

5.7 RQ7: How do SLMs compare with substantially larger language models across code generation, error detection, and tool-assisted settings?

This RQ synthesizes how the evaluated SLMs compare with two substantially larger language models across code generation, error detection, specialization, static-analysis-assisted detection, and iterative generation. All models are evaluated on the same EvalPlus benchmark and under the same execution-based reference used throughout the study whenever the corresponding experimental condition permits a direct comparison. This is not a causal scale experiment. The two larger models were purposively selected, and parameter count varies together with model family, training, architecture, backend, provider revision, and numerical precision. The results therefore characterize the evaluated models under a common experimental framework rather than isolate the effect of model scale.

5.7.1 Code Generation

The code-generation experiment provides the clearest evidence that the substantially larger models do not establish a separate upper performance range on these tasks. The three highest observed proportions of *Correct* solutions are all obtained by SLMs under the same EvalPlus evaluation harness: Muse Glimmer reaches 80.8%, GPT-OSS 79.0%, and Gemma 4 31B 78.7%. DeepSeek V4.1 Flash and MiMo V2.5 each reach 73.4%, which is also the result obtained by Qwen 2.5 Coder 7B.

Thus, selected SLMs do not merely fall within the same observed range as the larger baselines. Several obtain higher correctness proportions under the evaluated generation protocol despite a substantially smaller nominal parameter count. The strongest SLMs in this comparison contain approximately 20B to 31B parameters, whereas the two hosted baselines contain 310B and 552B parameters. These results do not establish that smaller models are generally superior, but they show that an approximately order-of-magnitude increase in parameter count does not guarantee a higher Correct proportion on the evaluated function-level tasks.

The absence of a simple scale ordering also appears within a single model family. Qwen 2.5 Coder 7B reaches 73.4% Correct, compared with 67.1% for the 32B configuration. This observation is consistent with the cross-family comparison: increasing parameter count alone is insufficient to explain the differences observed under this generation protocol.

Larger scale also does not eliminate structural or output-contract failures. DeepSeek produces approximately 5.9% Syntax outcomes, largely associated with responses that do not satisfy the expected function-level output structure. Most SLMs remain below approximately 3%, although Ornith is an important exception with a substantially higher Syntax rate. The comparison therefore indicates that instruction following and compliance with the expected generation interface remain relevant even for substantially larger models.

5.7.2 Hallucination Detection

The hallucination-detection results show a similar absence of a distinct performance range for the larger models. Muse Glimmer reaches 73.1% Accuracy and 53.8% Macro-F1 among the general-purpose SLMs. DeepSeek reaches 75.5% Accuracy and 51.0% Macro-F1, while MiMo reaches 51.5% Accuracy and 39.7% Macro-F1. Muse therefore approaches DeepSeek in Accuracy while exceeding both larger baselines in Macro-F1.

The comparison becomes more pronounced after specialization. Best Acc reaches 77.8% Accuracy, while Weighted MF1 reaches 54.2% Macro-F1. Both values exceed the corresponding aggregate results of the two general-purpose larger models. These improvements are obtained without increasing the parameter count of the underlying Gemma model, indicating that task specialization can alter detection capability substantially without relying on additional model scale.

The class-specific results also show that the main limitation identified in RQ2 persists for the substantially larger models. *Runtime Error* remains difficult across both groups. DeepSeek reaches 14.8% Runtime Recall and MiMo reaches 21.5%, while GPT-OSS reaches 32.9% and several other SLM configurations also fall within or above the large-model range. Thus, increasing scale from tens to hundreds of billions of parameters does not remove the difficulty of recog-

nizing Runtime failures under the evaluated protocol.

These results reinforce the distinction between generation and evaluation capability. Some SLMs are strong generators but weak detectors, while specialized SLMs improve detection substantially without increasing model size. The substantially larger baselines exhibit the same broader separation between these capabilities, indicating that the difficulty of recognizing generated-code failures is not resolved by parameter scale alone.

5.7.3 Tool-Assisted Error Detection

Static-analysis feedback likewise does not produce a clear advantage for the substantially larger models. The strongest effect remains the improvement in *Syntax Error* detection. Most SLMs reach 100% or near-100% Syntax Recall after receiving compilation feedback, and both larger models reach 100%. Once deterministic syntactic evidence is supplied explicitly, the evaluated SLMs are therefore generally able to exploit it as effectively as the substantially larger baselines.

The aggregate results further show that selected SLMs remain highly competitive after external evidence is introduced. Muse Glimmer reaches approximately 79% Accuracy and 71% Macro-F1, while Gemma 4 31B reaches 77% Accuracy and 66% Macro-F1. The specialized Weighted MF1 detector reaches approximately 76% Accuracy and 68% Macro-F1. DeepSeek reaches approximately 76% Accuracy and 64% Macro-F1, while MiMo reaches 47% for both metrics. Thus, static feedback does not create a new performance separation in favor of the substantially larger models. Several SLM conditions equal or exceed the observed aggregate results of the hosted baselines.

The Runtime limitation also persists after tool integration. DeepSeek decreases from 14.8% to approximately 10% Runtime Recall, while MiMo decreases from 21.5% to approximately 9%. Similar reductions occur for many SLMs. The difficulty identified in RQ5 is therefore shared across the evaluated scale range: incomplete Pyright evidence is not reliably translated into an explicit *Runtime Error* prediction even by models with hundreds of billions of parameters.

The indirect effect of static feedback on classes not evaluated by the tools is also present at larger scale. DeepSeek decreases Correct Recall while increasing Functional Recall, and MiMo presents an even clearer redistribution, with Correct Recall decreasing by approximately 10 percentage points while Functional Recall increases by approximately 10 points. The Correct-to-Functional shifts observed among several SLMs in RQ5 are therefore not exclusive to compact models. Both groups can interpret static diagnostics as broader evidence about program correctness, even when the tools provide no direct information about functional behavior.

Together, these results suggest that the effect of tool assistance depends more strongly on the type and coverage of the supplied evidence and on the model's learned decision behavior than on parameter count alone. Direct compilation evidence is successfully incorporated across model scales, whereas partial Runtime evidence remains difficult to interpret consistently.

5.7.4 Tool-Assisted Iterative Code Generation

The iterative generation experiment presents a somewhat different pattern. Among the eleven SLMs, six increase their proportion of *Correct* solutions relative to RQ1, while five decrease it. In contrast, both substantially larger models improve. DeepSeek increases Correct by 7.3 percentage points and MiMo by 6.6 points. RQ6 provides the corresponding paired analysis for these two models.

This difference indicates a more consistently positive response to the iterative protocol for the two larger models evaluated in this study. However, the improvement does not establish a higher final correctness range. Gemma 4 31B reaches approximately 82% Correct after the iterative protocol, compared with approximately 81% for DeepSeek and 80% for MiMo. GPT-OSS also reaches approximately 80%. The highest final Correct proportion therefore remains associated with an SLM despite the more consistent direction of change observed for the two substantially larger models.

The error distributions also reveal similarities across scales. Runtime and Syntax failures frequently decrease during iterative generation for both SLMs and the larger models. Functional behavior remains more model-dependent, and several SLMs achieve correctness gains through reductions that extend beyond the error categories directly targeted by the tools. Qwen 2.5 Coder 32B, for example, obtains the largest SLM increase in Correct while substantially reducing Functional failures, despite the absence of direct functional evidence from compilation or Pyright.

The more consistent improvement observed for DeepSeek and MiMo therefore appears to concern the response to the complete iterative protocol rather than the maximum final performance reached. With only two substantially larger models, this pattern is insufficient to establish that model scale causes more reliable use of feedback. It instead motivates the narrower observation that both hosted baselines respond positively in this experiment, whereas the SLM responses are divided between improvement and regression.

Across the evaluated settings, increasing parameter count does not produce a uniform performance advantage. Under the same benchmark and execution-based evaluation framework, the three highest code-generation Correct proportions are obtained by SLMs, specialized SLMs reach or exceed the larger baselines in hallucination detection, and selected SLMs also equal or surpass their aggregate performance when static-analysis evidence is provided. Runtime detection remains difficult at both scales, and the Correct-to-Functional redistribution caused by tool feedback also appears in the substantially larger models. The iterative protocol is the main setting in which the two larger baselines exhibit a more consistently positive direction of change, yet their final correctness remains within the range reached by the strongest SLMs. The results therefore do not support a general performance ordering based on parameter count and instead show that specialization, error type, and the ability to interpret and act on external evidence are important dimensions of model reliability under the evaluated protocols.

6 Threats to Validity

This study has several limitations that should be considered when interpreting the results. We organize them according to construct, internal, external, conclusion, and reliability threats, followed by ethical considerations.

6.1 Construct Validity

The experiments use fixed prompting configurations for code generation, hallucination detection, and tool-assisted interaction. This choice is intentional and preserves comparability by avoiding model-specific prompt optimization. Consequently, the results characterize model behavior under a common prompting protocol rather than the maximum performance that could be achieved through prompt search, few-shot examples, explicit reasoning strategies, or model-specific instructions. Such optimization could improve individual models, but would introduce an additional source of variability outside the scope of the present comparison.

The proposed hallucination taxonomy deliberately operationalizes generated-code failures through observable execution outcomes: *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*. This outcome-oriented formulation extends evaluation beyond binary correctness while allowing labels to be grounded automatically through compilation and execution. It is not intended as an exhaustive taxonomy of every possible manifestation of hallucination in software engineering. The conclusions therefore concern these observable outcome categories rather than other properties such as security, performance, maintainability, dependency misuse, or repository-level API consistency.

The multilabel formulation introduces an inherent asymmetry between classes. *Runtime Error* and *Functional Error* may co-occur because different test cases can expose different failures, whereas *Syntax Error* prevents execution and *Correct* is mutually exclusive with all error labels. Consequently, a syntactically invalid solution cannot also be evaluated for potential Runtime or Functional failures. This behavior follows the execution semantics of the evaluation pipeline and should be considered when comparing class prevalence.

For visualization, the confusion matrices reduce each multilabel reference and prediction to one primary class using the declared priority *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*. This reduction is used only for visualization and does not replace the multilabel metrics used in the quantitative evaluation.

Detection performance also depends on compliance with the required structured-output format. Responses that cannot be parsed are retained in the evaluation and mapped to an empty label set rather than discarded. Consequently, the reported detection metrics reflect both error-recognition capability and adherence to the requested output protocol. This decision avoids selectively removing model failures, but aggregate metrics cannot distinguish a classification error from a case in which the intended classification was expressed in an invalid format.

Finally, the *Correct* label is determined by the available EvalPlus test suites. Although EvalPlus extends the original HumanEval and MBPP tests, passing all available cases does not constitute a formal proof of semantic correctness. A generated solution may still fail on an input that is not represented by the benchmark. *Correct* should therefore be interpreted as correctness with respect to the available evaluation suite.

6.2 Internal Validity

The generation pipeline includes response normalization and code extraction before compilation and execution. Some responses may contain meaningful code but fail to satisfy the expected output contract, causing the evaluation pipeline to produce a *Syntax Error*. This creates an interaction between model behavior and the extraction procedure. However, compliance with the requested function-level output is also part of the evaluated generation task. Models that frequently return incompatible structures cannot be treated as equivalent to models that consistently produce executable responses.

The evaluated models are executed under different numerical configurations. Local models used for the initial generation experiments were generally quantized to make the experimental matrix computationally feasible, while some variants and the hosted large models were executed under different precision settings. A limited comparison between quantized and non-quantized Gemma variants indicates that numerical configuration can affect detection performance, but this factor is not isolated systematically for every model family. Some cross-model differences may therefore reflect inference configuration in addition to architecture and training.

The specialized detectors are based on Gemma-family models and use a specific QLoRA training pipeline. The observed improvements demonstrate that specialization can be effective in the evaluated setting, but they do not establish that equivalent gains would necessarily occur for every SLM family or adaptation method. Similarly, differences among the Accuracy-oriented, Macro-F1-oriented, and balanced configurations reflect the particular training objectives, hyperparameter search space, and dataset construction used in this study.

The knowledge-distillation process introduces a potential source of supervision noise. Reference classes are always derived from execution, but the natural-language explanations used as auxiliary training targets are generated by three teacher models. These rationales may contain incomplete or inaccurate reasoning even when associated with a correct reference label. Teacher models do not determine or modify the reference classes, and the quantitative evaluation depends only on predicted labels, but the textual quality of the rationales was not independently validated and may influence fine-tuning behavior.

The tool-assisted detection experiment introduces static-analysis evidence as an additional component of the model input. Although the candidate code, task, classification instructions, decoding configuration, output schema, and reference labels are otherwise preserved, the experiment cannot fully separate the semantic contribution of the diagnostics from the effect of introducing an additional external-analysis message into the prompt. This is particularly relevant to the shifts observed between *Correct* and *Functional Error*, since neither Python compilation nor Pyright directly evaluates functional

correctness. Isolating this effect would require a control condition with an equivalent prompt structure but without informative diagnostic content.

The interpretation of Pyright feedback also depends on the limited coverage of the selected diagnostic mapping. As shown in RQ4, only a small fraction of the execution-derived Runtime failures are identified statically. Most Runtime cases therefore receive no mapped Runtime diagnostic. The prompt explicitly states that the absence of a diagnostic does not establish the absence of an error, but the experiment does not isolate how strongly individual models rely on this negative evidence. The reduction in Runtime Recall observed for many models in RQ5 is consistent with this possibility, but does not by itself establish the mechanism.

6.3 External Validity

The experiments are restricted to Python and to the HumanEval and MBPP tasks included in EvalPlus. Although this setting provides a controlled environment with executable reference tests, the benchmark primarily represents isolated function-level programming tasks and only partially reflects real software development. Production development commonly requires reasoning across multiple files, existing implementations, external dependencies, APIs, build systems, configuration, persistent state, repository conventions, and interactions among components. The hallucinations and failure patterns observed in such environments may differ substantially from those identified in the present benchmark. Consequently, the results should be interpreted as evidence about function-level code generation and evaluation rather than as a direct characterization of model reliability in complete software-development environments. Repository-level evaluation with executable project tests is necessary to determine whether the observed relationships among generation, detection, tool use, and revision generalize to realistic software projects.

The evaluated model set covers multiple architectures, sizes, and code-oriented models, but remains a sample of a rapidly changing model landscape. The conclusions should therefore not be generalized to all SLMs or all substantially larger language models. In particular, RQ7 includes only two substantially larger hosted baselines. The comparison is useful for examining whether a large parameter gap necessarily corresponds to a different observed performance range, but broader claims about scaling would require a larger and more systematically sampled set of models.

Nominal parameter count should also be interpreted only as an approximate indicator of scale. The evaluated systems differ in architecture, training procedure, active computation, numerical precision, serving backend, and other factors that are not controlled by parameter count. The results of RQ7 therefore do not measure computational efficiency and should not be interpreted as a causal relationship between the number of parameters and model capability.

The static-analysis conclusions are similarly specific to the techniques and diagnostics evaluated in this study. Python compilation provides direct evidence of syntactic validity, while the selected Pyright diagnostics cover only a subset of the Runtime failures observed during execution. The weak Runtime coverage of this configuration should not be generalized to static analysis as a whole. Other approaches, including abstract interpretation, symbolic execution, interprocedural analysis, dynamic analysis, or different diagnostic mappings, could provide different evidence and produce different interactions with language models.

6.4 Conclusion Validity

RQ1 evaluates one generated solution per model and task. The reported correctness proportions therefore characterize the observed generation run and do not estimate pass@k or within-model sampling variability. Repeated generations could be used in future work to characterize the stability of the observed proportions under different sampled outputs, but this is distinct from the model comparison investigated here.

The inferential analysis is intentionally restricted to comparisons that directly support the corresponding research questions. RQ1 and RQ2 are descriptive. RQ3 uses paired task-level bootstrap intervals to quantify changes in Accuracy and Macro-F1 between the selected specialized detectors and their corresponding base condition. RQ4 and RQ5 are descriptive because their purpose is to characterize detection coverage and changes in class-specific behavior. RQ7 is also descriptive because model scale is confounded with model family, architecture, training, numerical precision, and backend.

RQ6 compares two complete generation protocols rather than isolating the causal contribution of static analysis. The original generation condition uses a temperature of 1.0 and a single generation, whereas the iterative condition uses a temperature of 0.0 together with static feedback, self-review, and repeated revision. Even the initial candidate produced under the iterative protocol is therefore generated under a different decoding condition from the RQ1 candidate. Changes between RQ1 and RQ6 may consequently reflect the combined effect of decoding configuration, new generation, self-review, external evidence, and revision.

RQ6 reports descriptive changes for all evaluated models. Exact two-sided McNemar tests are used only where paired task-level observations from both protocols are available. In the retained data, this applies to the two hosted models. The SLM changes should therefore be interpreted as descriptive differences between the two evaluated protocols rather than inferentially established improvements or regressions.

The reference distribution is also strongly imbalanced, with *Syntax Error* particularly rare. Some class-specific analyses consequently contain small denominators. For example, the held-out static-analysis comparison contains only 17 Syntax and 79 Runtime reference cases, and some self-detection results are based on relatively small error subsets. To reduce the risk of misleading interpretation, the paper reports relevant denominators and considers Macro-F1 and per-class Recall alongside Exact-set Accuracy.

6.5 Reliability

The experiments involve multiple stages, including generation, output normalization, compilation, execution, knowl-

edge distillation, fine-tuning, static analysis, detection, and iterative tool-assisted interaction. Errors introduced in one stage could propagate to later analyses. To reduce this risk, the same execution-derived reference labels are reused across detection experiments, dataset partitions are performed at the task level to prevent overlap between training, validation, and test tasks, and model outputs, execution outcomes, and tool diagnostics are explicitly logged.

A temperature of 0.0 is used during detection, knowledge-distillation inference, tool-assisted detection, and iterative generation to reduce sampling variability. This setting should not be interpreted as guaranteeing deterministic execution, particularly for hosted models whose serving infrastructure is outside our control. Exact reproduction may also depend on the continued availability of the same hosted model revisions and provider backends.

6.6 Ethical Considerations and Responsible Use

This computational study uses public programming benchmarks and generated code and involves neither human participants nor personal data. Candidate programs are executed in isolated subprocesses with timeouts rather than on user systems. Requests sent to hosted models contain public benchmark descriptions and generated benchmark solutions rather than private source code.

The proposed detector should be interpreted as an error-detection aid rather than a formal verification mechanism. False negatives may create unwarranted confidence, while false positives may lead developers to discard valid code. Similarly, successful compilation or the absence of a Pyright diagnostic provides no guarantee of functional correctness, and passing the available EvalPlus tests does not establish correctness for unseen inputs. The approach should therefore complement, rather than replace, compilation, testing, static or dynamic analysis, security review, and human code review. Model-generated or tool-derived explanations should be treated as diagnostic evidence that may require independent verification before use in consequential software.

7 Conclusion

This study investigated the reliability of Small Language Models for code generation beyond the traditional question of whether generated code passes a test suite. We examined whether SLMs can generate correct code, recognize hallucinations in generated programs, identify failures in their own outputs, benefit from task specialization, interpret static-analysis evidence, and act on this evidence during iterative revision. To support this analysis, generated-code behavior was modeled through four execution-grounded outcomes: *Syntax Error*, *Runtime Error*, *Functional Error*, and *Correct*. This formulation makes it possible to distinguish generation capability from the ability to assess and revise generated code.

The generation results show that SLMs can solve a substantial portion of the evaluated programming tasks, but generation quality alone does not characterize reliability. Models with strong generation performance are not necessarily strong error detectors, and models that produce relatively few failures do not necessarily recognize the failures they do produce. *Runtime Error* is particularly difficult to identify, whereas *Functional Error* is generally easier for the evaluated detectors. Self-detection results reinforce this separation, showing that producing a solution and evaluating that solution are distinct capabilities.

Specialization substantially improves error detection in the evaluated Gemma setting. The Accuracy-oriented detector increases Exact-set Accuracy by 32.0 percentage points, with a paired bootstrap interval of [25.0, 39.2], while the Macro-F1-oriented detector increases Macro-F1 by 17.2 points [11.9, 22.3]. The improvements are not uniform across classes. Accuracy-oriented training favors recognition of *Correct*, whereas Macro-F1-oriented training produces a more balanced class profile. The results therefore show that specialization can significantly improve detection without increasing model size, but the optimization objective determines which failure types the detector becomes more capable of recognizing.

The comparison with static analysis reveals a clear division of strengths. Python compilation provides direct and reliable verification of syntactic validity, while most evaluated SLMs are substantially less consistent when asked to infer the same property. Runtime failures present the opposite pattern. The selected Pyright diagnostic mapping identifies only a small fraction of the Runtime failures observed during execution, whereas several SLMs identify a larger portion of these cases. Deterministic tools are therefore particularly effective for properties that can be checked directly, while model-based detection can provide additional coverage for failures that are difficult to expose statically.

Providing static-analysis evidence directly to the detectors does not result in a uniform improvement. Compilation feedback raises Syntax Recall to near-perfect levels for almost every evaluated configuration, indicating that models can effectively exploit a direct and highly informative signal. Runtime behaves differently. Runtime Recall decreases in most configurations after Pyright feedback is introduced, which is consistent with the limited Runtime coverage of the static analysis used in the experiment. The feedback also alters predictions for classes that are not evaluated by the tools. In particular, many models shift predictions from *Correct* toward *Functional Error*, even though compilation and Pyright provide no direct evidence of functional correctness. External evidence therefore does not simply add information to the detector. Its effect depends on how the model interprets both the presence and absence of tool diagnostics.

The iterative generation experiment shows that interpreting evidence for classification and acting on evidence during revision are also different capabilities. Six of the eleven evaluated SLMs finish the iterative protocol with a higher proportion of *Correct* solutions, while five finish with fewer. Runtime failures decrease for most models, even though Runtime classification frequently deteriorates when similar static evidence is used in the detection experiment. The source of improvement also differs across models. Some gains are primarily associated with removing Syntax failures, whereas others involve reductions in Runtime or Functional failures. Conversely, some models improve specific failure categories

while introducing new errors elsewhere. Additional feedback and revision opportunities therefore do not guarantee a better final program.

The comparison with two substantially larger language models further shows that parameter count alone does not determine performance under the evaluated conditions. Under the same benchmark and execution-based evaluation framework, the three highest Correct proportions in the original generation experiment are obtained by SLMs. Specialized SLMs also reach or exceed the larger baselines in hallucination detection, and selected SLM configurations remain competitive when static-analysis evidence is introduced. The two substantially larger models respond positively to the iterative generation protocol, while the SLM results are more heterogeneous, but their final correctness remains within the range achieved by the strongest SLMs. These observations do not establish that smaller models are generally superior. They show that substantially increasing parameter count does not, by itself, resolve the reliability limitations observed in code generation and evaluation.

Taken together, the results indicate that reliable AI-assisted programming cannot be reduced to a single dimension of model capability. Code generation, error recognition, self-evaluation, interpretation of external evidence, and code revision behave as distinct capabilities and may vary substantially within the same model. Increasing model size does not eliminate these differences, and providing additional evidence does not guarantee that the evidence will be interpreted correctly. The strongest results instead support a complementary view in which specialized models, deterministic software-engineering tools, and iterative revision mechanisms address different parts of the reliability problem.

7.1 Future Work

The most important direction for future work is to evaluate whether these findings remain valid in tasks that more closely represent real software development. EvalPlus provides a controlled and execution-grounded environment, but its tasks are predominantly isolated programming problems at the function level. Real development requires reasoning over substantially broader contexts, including multiple files, existing implementations, APIs, dependencies, build systems, configuration files, tests, repository conventions, and interactions between components. Hallucinations that arise in these environments may differ substantially from those observed in isolated function completion.

A natural extension is therefore to construct or adopt repository-level benchmarks in which models operate on realistic software projects and their outputs can be validated through compilation, project tests, integration tests, and execution. Such an environment would allow the current outcome-oriented classification to be extended to failures involving dependencies, APIs, interfaces, project structure, and repository context. It would also make it possible to evaluate whether the separation observed in this study between generation, detection, and revision persists when models must reason about a larger and more realistic software state.

Repository-level evaluation is also important for studying the practical value of external evidence. In realistic projects, static analysis can provide information beyond syntax and local type inconsistencies, while build systems, test suites, execution traces, coverage information, and project-specific diagnostics can expose different aspects of program behavior. Future work should investigate not only which tools detect useful information, but also which forms of evidence language models can interpret correctly and convert into successful revisions.

The tool-assisted detection experiment also motivates a more controlled investigation of how external evidence changes model behavior. A future experiment could introduce a control condition with the same additional prompt structure but without informative diagnostics. This would help separate the semantic contribution of the tool output from the effect of simply adding an external-analysis message to the model context, particularly for the unexpected redistribution between *Correct* and *Functional Error*.

The iterative generation protocol should likewise be decomposed into its individual components. Future experiments could begin from the same initial candidate and separately introduce self-review, static-analysis feedback, and repeated revision. Such ablations would make it possible to determine whether improvements originate from the diagnostic information itself, from additional opportunities to reconsider the solution, or from their interaction.

Finally, specialization should be evaluated across additional model families and more realistic error distributions. Comparing rationale-based supervision with label-only training, evaluating class-specific or hierarchical detectors, and separating structured-output compliance from error-recognition capability could clarify which aspects of the current specialization pipeline are responsible for the observed improvements. Extending these experiments to repository-level development environments would provide a stronger test of whether specialized SLMs can serve as practical components of reliable AI-assisted software-engineering workflows.

Overall, the present results provide evidence that reliability can be improved by combining complementary capabilities, but function-level correctness represents only one part of real software development. Evaluating these mechanisms in complete repositories, under realistic development constraints and with executable project-level validation, is the necessary next step for determining whether the observed benefits translate to practical software-engineering environments.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 2623–2631.

Andriushchenko, G., Garaev, R., Rvanova, L., Shelmanov, A., and Ivanov, V. V. (2026). Efficient hallucination detection in automatic code generation. In Liakata, M., Moreira, V. P., Zhang, J., and Jurgens, D., editors, *Findings of*

the Association for Computational Linguistics: ACL 2026, pages 43197–43220, San Diego, California, United States. Association for Computational Linguistics.

Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., et al. (2021). Program synthesis with large language models.

Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code.

Chen, X., Lin, M., Schärli, N., and Zhou, D. (2024). Teaching large language models to self-debug. In *The Twelfth International Conference on Learning Representations*.

DeepSeek-AI (2026). DeepSeek-V4.1-Flash: Pushing the limits of KV cache compression. Technical report.

Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems*, volume 36.

Dou, S., Jia, H., Wu, S., Zheng, H., Wu, M., Tao, Y., Zhang, M., Chai, M., Fan, J., Xi, Z., Zheng, R., Wu, Y., Wen, M., Gui, T., Zhang, Q., Qiu, X., and Huang, X. (2025). What's wrong with your code generated by large language models? an extensive study. Accepted by Science China Information Sciences, according to the arXiv record.

Gemma Team, El Abd, S., Aggarwal, V., Algayres, R., Andreev, A., Bachem, O., et al. (2026). Gemma 4 Technical Report.

Gemma Team, Kamath, A., Ferret, J., Pathak, S., Vieillard, N., Merhej, R., et al. (2025). Gemma 3 Technical Report.

GLM-4.5 Team, Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., et al. (2025). GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models.

Hsieh, C.-Y., Li, C.-L., Yeh, C.-k., Nakhost, H., Fujii, Y., Ratner, A., Krishna, R., Lee, C.-Y., and Pfister, T. (2023). Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 8003–8017, Toronto, Canada. Association for Computational Linguistics.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.

Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., et al. (2024). Qwen2.5-Coder Technical Report.

Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. C. H. (2022). CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 35. Curran Associates, Inc.

Lee, Y., Song, J. Y., Kim, D., Kim, J., Kim, M., and Nam, J. (2025). Hallucination by code generation LLMs: Taxonomy, benchmarks, mitigation, and challenges.

Liu, J., Xia, C. S., Wang, Y., and Zhang, L. (2023). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc.

Ludan, J. M., Meng, Y., Nguyen, T., Shah, S., Lyu, Q., Apidianaki, M., and Callison-Burch, C. (2023). Explanation-based finetuning makes models more robust to spurious cues. In Rogers, A., Boyd-Graber, J., and Okazaki, N., editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4420–4441, Toronto, Canada. Association for Computational Linguistics.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B. P., Hermann, K., Welleck, S., Yazdanbakhsh, A., and Clark, P. (2023). Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.

Meta Superintelligence Lab (2026). Muse Glimmer 30B Model Card. Hugging Face model card.

Microsoft (2026). Pyright: Static type checker for python. Software repository.

Mistral AI (2025). Introducing Devstral 2 and Mistral Vibe CLI. Official model release.

OpenAI, Agarwal, S., Ahmad, L., Ai, J., Altman, S., Applebaum, A., et al. (2025). gpt-oss-120b & gpt-oss-20b model card.

Ornith Team (2026). Ornith-1.5: From self-scaffolding to self-improvement. Official model report.

Pavel, M., Petrovic, N., Mazur, L., Zolfaghari, V., Pan, F., and Knoll, A. (2025). Hallucination in LLM-based code generation: An automotive case study.

Python Software Foundation (2026). Built-in function `compile`. Python 3.13 documentation.

Qwen Team (2025). Qwen3-Coder: Agentic coding in the world. Official model release.

Qwen Team (2026). Qwen3.8-Max: A new bar for coding and cowork. Official model release.

Rastogi, A., Yang, A., Jiang, A. Q., Liu, A. H., Sablayrolles, A., et al. (2025). Devstral: Fine-tuning language models for coding agent applications.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., and Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36. Curran Associates, Inc.

Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., and Song, D. (2025). CodeHalu: Investigating code hallucinations in LLMs via execution-based verification. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, pages 25300–25308. AAAI Press.

Xiaomi MiMo Team (2026). MiMo-V2.5. Official model card.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., et al. (2025). Qwen3 Technical Report.

Z.ai (2025). GLM-4.7-Flash model card. Hugging Face model card.